



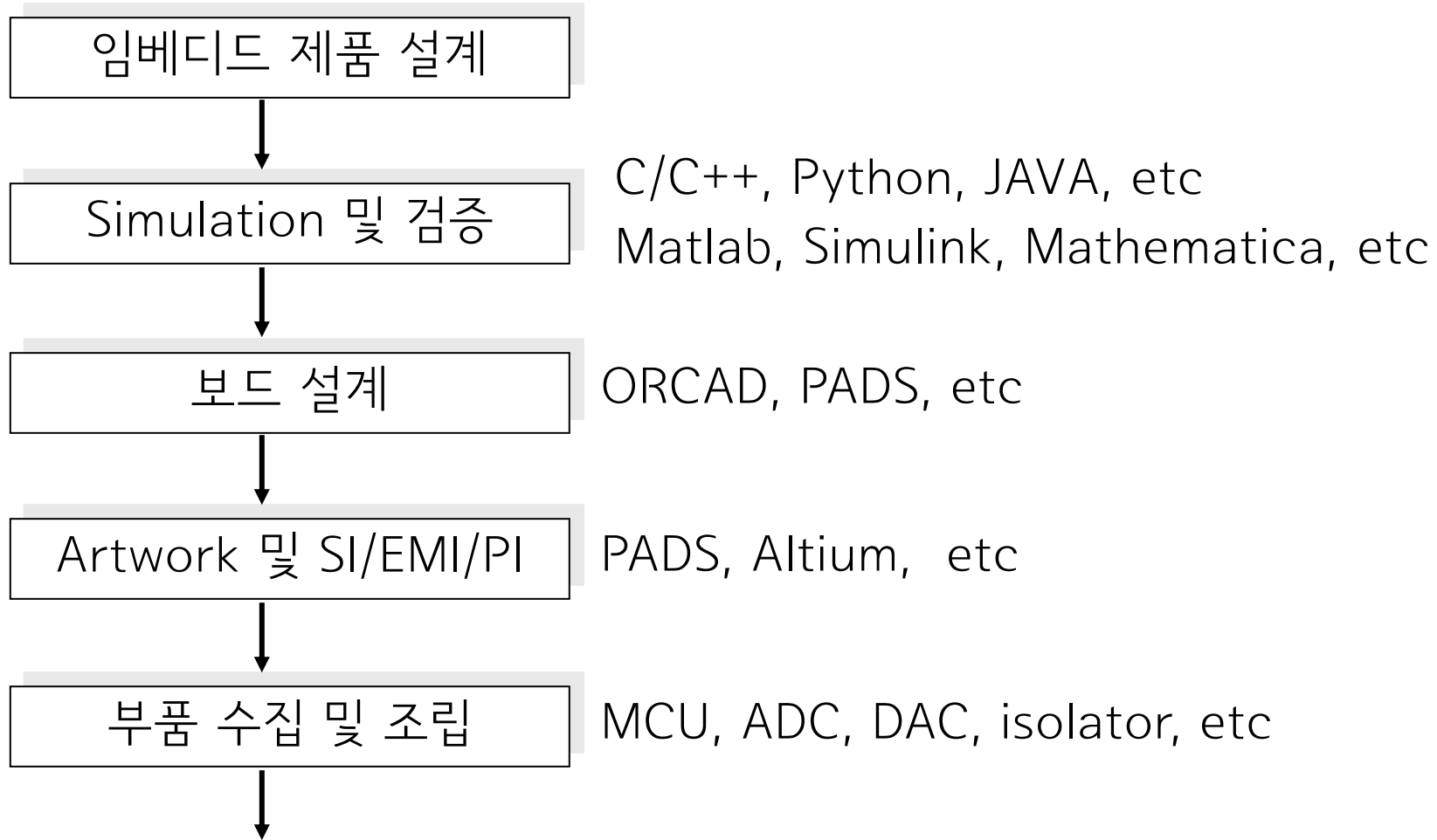
서울시립대학교

STM32 Cortex-M을 활용한 마이크로프로세서응용실습

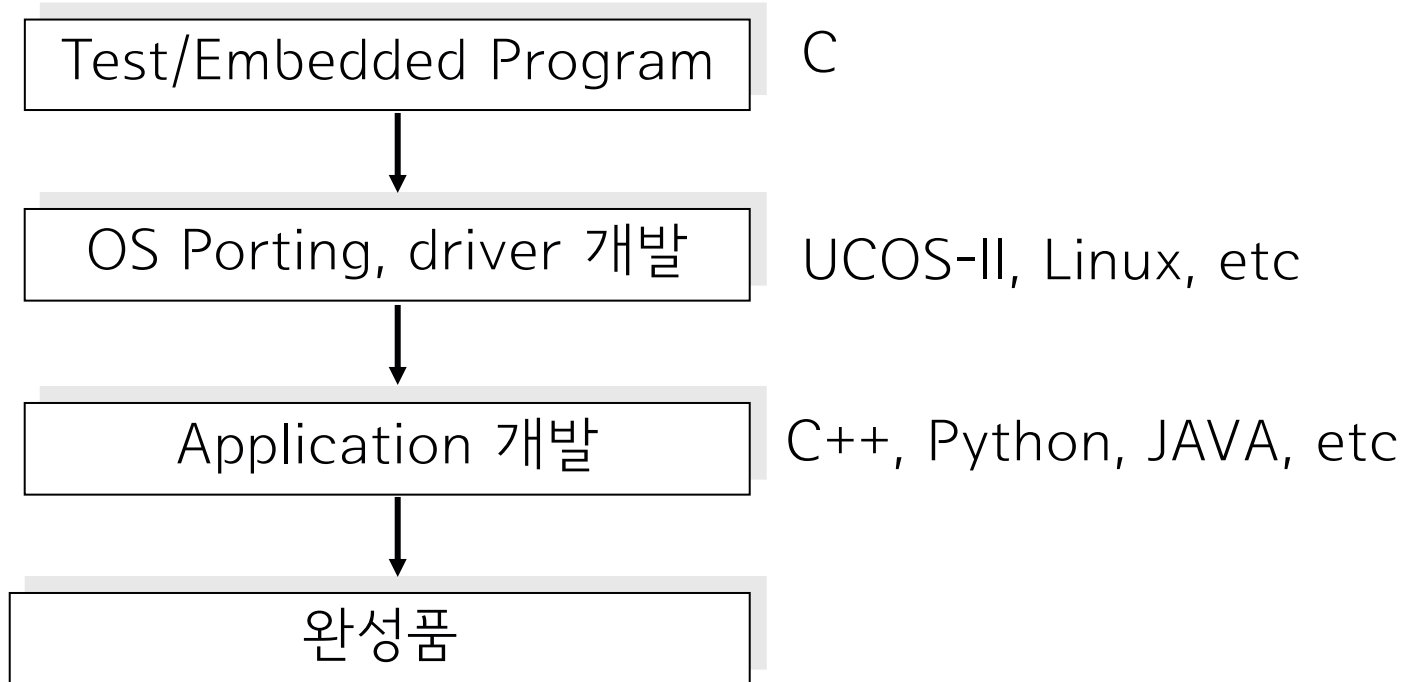
Lec 1 - Introduction (C programming review)

전력전자연구실 교수 이 승 환



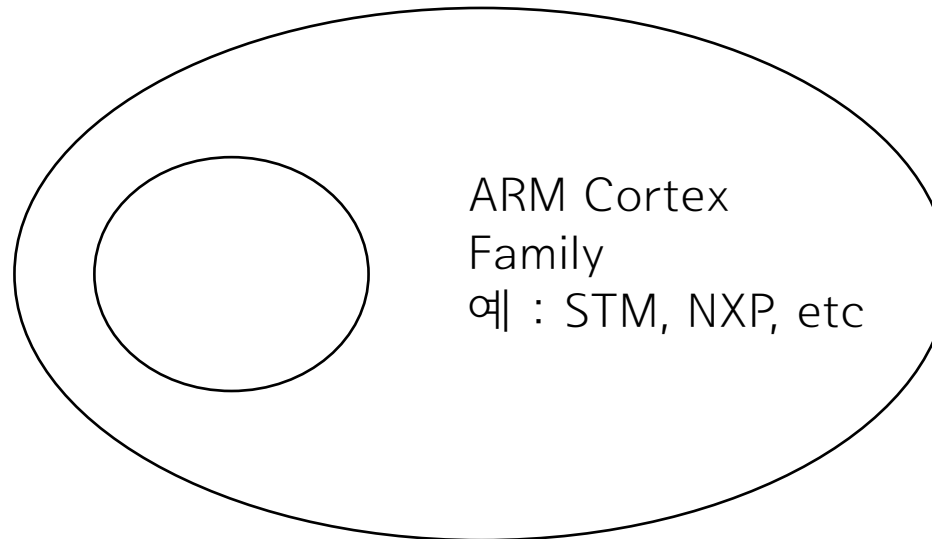


C언어 소개와 개발용 언어 소개.



CPU, MCU, DSP 개념과 구조 - 1.

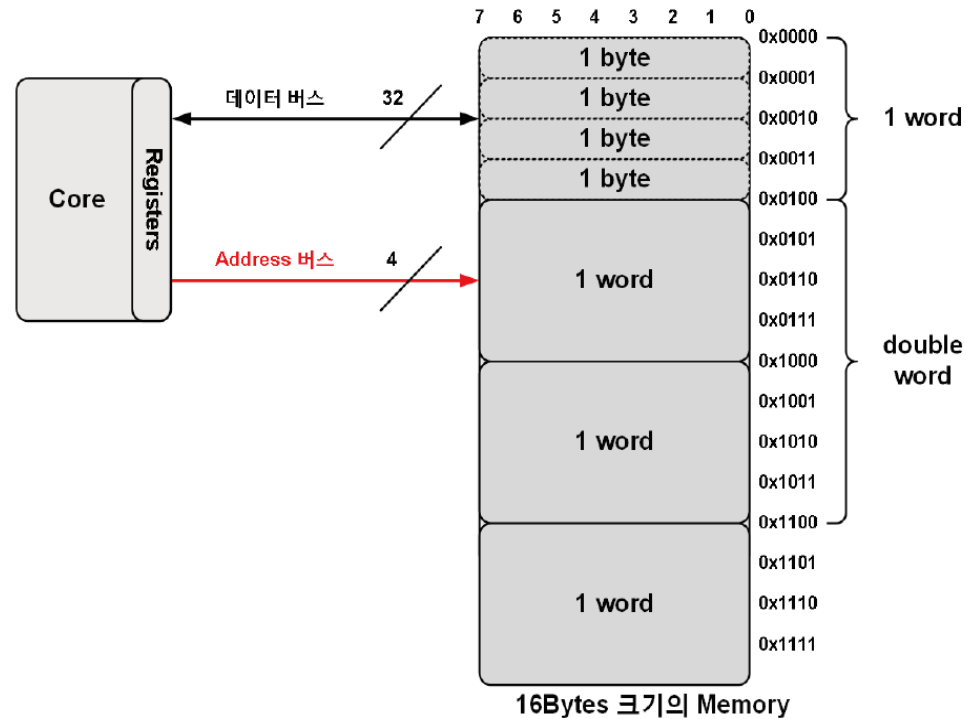
제조사 고유의 processor 개발/판매
예 : Z80, 8051, AVR, PowerPC, TI, etc



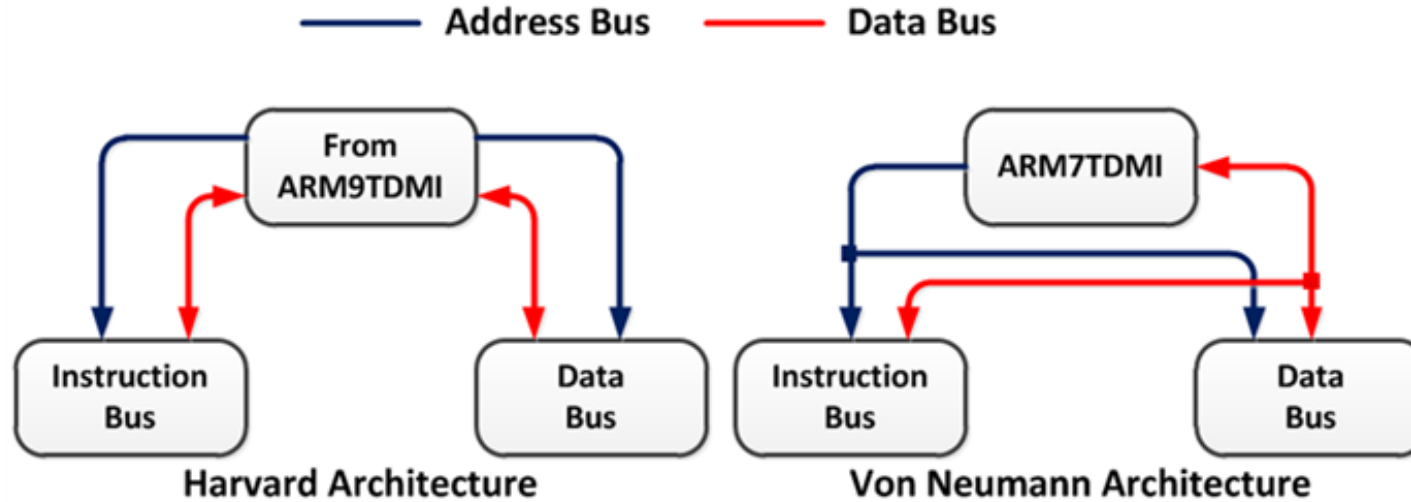
✓ Processor의 주요 내부 구조.

- Core :
 - 명령어를 해석하고, 그에 따른 결과로 각종 주변 장치들에게 명령을 하고, 데이터를 전송하거나 요구함.
- Interrupt Controller :
 - 비동기적으로 발생하는 event를 우선 순위에 맞게 core에게 전달함. Core-M family는 NVIC(Nested Vector Interrupt Controller)
- 주변 장치(peripheral) :
 - 외부 부품들과 연결하기 위한 port들, 예를 들면, UART, I2C, SPI, I2S, 등등과 같은 연결(interface)관련 규칙(사양, Spec.)을 구현한 회로. 이들 연결 설정과 제어는 해당 주변 장치의 controller에 대한 register 값을 설정하는 방식을 이용함.
- DMA(Direct Memory Access) :
 - Core의 직접적인 간섭 없이 memory를 포함한 주변 장치들 사이의 데이터 교환 방법. Core의 부하를 덜어주어 실행 속도를 개선할 수 있음.

- ✓ Processor 내부의 성분들과 상호 bus로 연결되어 필요한 명령어와 데이터를 전송.
 - Memory에는 8bits 즉, 1byte 단위로 번지가 할당되어 저장됩니다.



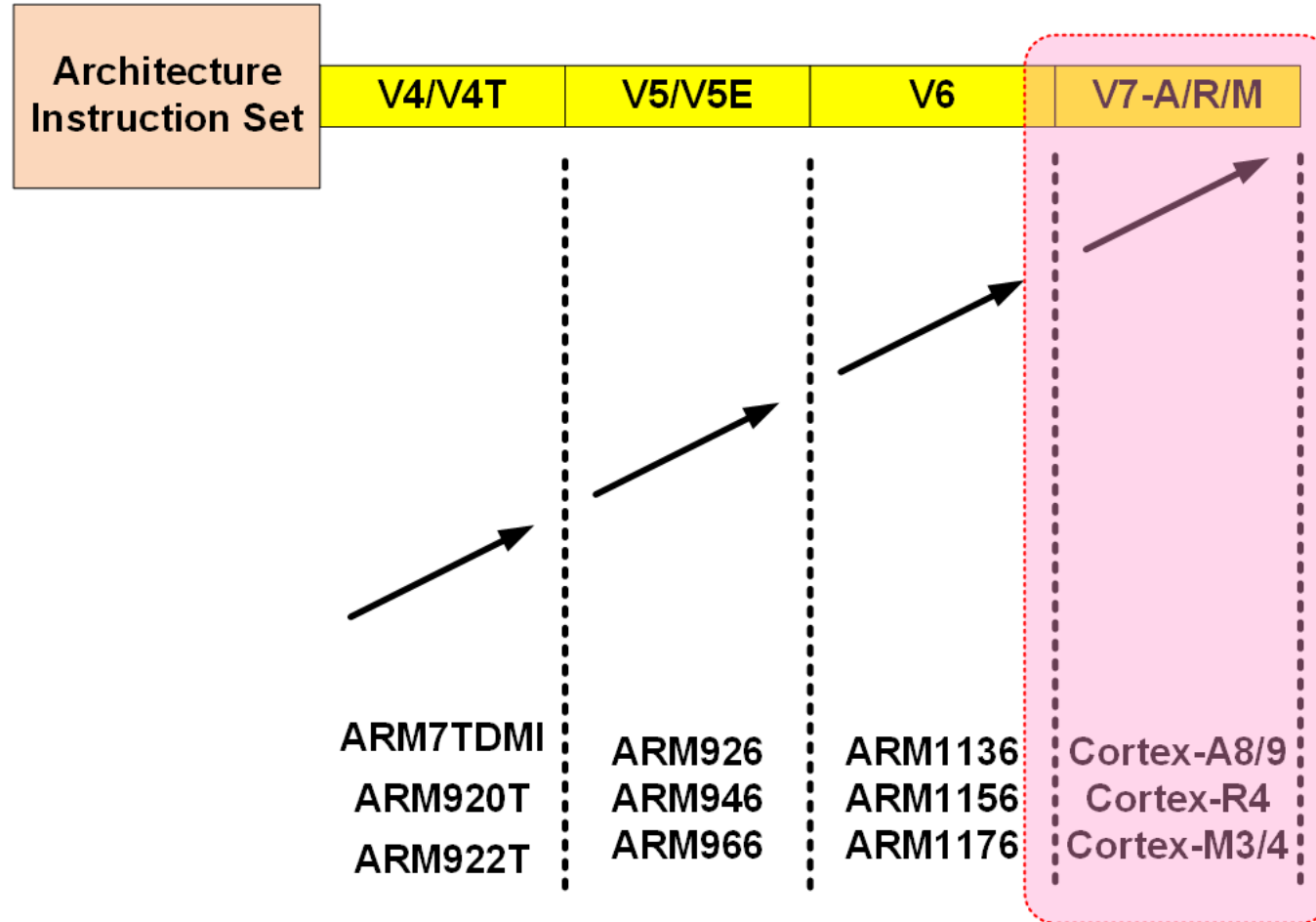
[그림 2.1-1] memory와 번지(address)의 개념.



- ✓ Von Neumann 구조 :
 - 메모리 공간에서 명령어와 데이터를 동시에 access 할 수 없다.
- ✓ Harvard 구조 :
 - 명령어와 데이터를 동시에 access 할 수 있다.
- ✓ ARM9 이후 버전은 고성능을 얻기 위하여 Harvard 구조를 사용.

- ✓ 여타 다른 Core와 비교하여 상대적으로 저전력, 작은 die size와 고성능을 발휘하도록 설계. → Mobile product에 적합.
- ✓ 지적 재산권(IP)관련 business만 수행.
 - 여러 Chip vendor에서 관련 processor 개발에 채택.
 - 가격의 하락, 동일한 Core로 인한 S/W 호환성, 단종 문제 해결.
 - 2010년 3분기 기준 license 계약 실적.
 - ARM7 family : 173 licenses.
 - ARM9 family : 269 licenses.
 - ARM11 family : 76 licenses.
 - Cortex-A family : 33 licenses. (ARM7, 9, 11가다가 13 대신 Cortex라는 이름으로 변경)
 - Cortex-R family : 17 licenses.
 - Cortex-M family : 51 licenses.
- ✓ 2011년 기준, 전세계 mobile product의 90% 이상이 ARM Core 채택.

- ✓ Architecture는 해당 core에서 사용할 수 있는 instruction set을 의미.



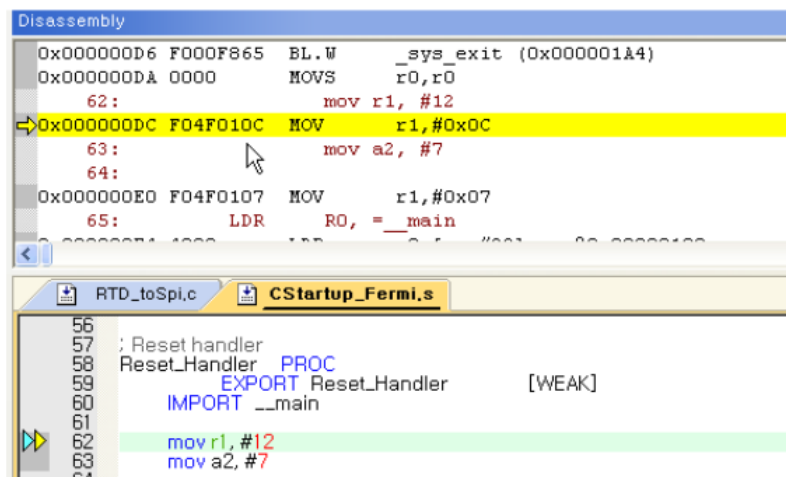
- ✓ Instruction set이란 해당 core에서 사용할 수 있는 assembly level의 명령어 집합들을 의미.
 - CISC(Complex Instruction Set Computer) 구조.
 - RISC(Reduced Instruction Set Computer) 구조.
- ✓ ARMv7-M architecture : RISC 구조, MMU(Memory Management Unit) 지원하지 않음.
 - Thumb-2 instruction set : 32-bit 또는 16-bit.
 - Thumb instruction set : 16-bit.
 - ARM instruction set 지원하지 않음. → 다른 ARM core와 호환성 문제!!!
 - 많은 새로운 명령어들 추가 :
 - hardware 나눗셈 명령어(예: udiv, sdiv) 지원.
 - 고속화된 여러 곱셈 명령어들(예: mul, mla, mls, smlal, smull, umlal, umull).
 - 비정렬 데이터 접근도 지원.

C/C++ Language

unsigned char ind=12;

C/C++ compiler

Assembly Language



Assembler

Encoding T2 ARMv7-M

MOV{S}<C>.W <Rd>, #<const>

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	0	0	0	1	0	S	1	1	1	1	0	imm3														

d = UInt(Rd); setflags = (S == '1'); (imm32, carry) = ThumbExpandImm_C(i:imm3:imm8, APSR.C);
if d IN {13,15} then UNPREDICTABLE;

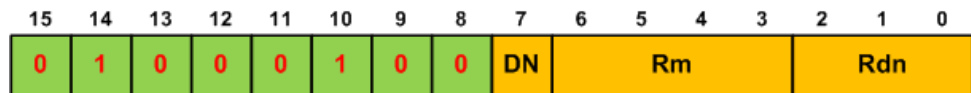
Hex-Code : Binary Code

F04F010C : 11110 0 0 0010 0 1111 0 000 0001 00001100

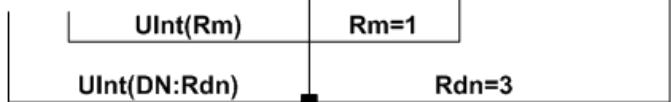
Memory

ADD<c> <Rdn>, <Rm> ; Thumb instruction set

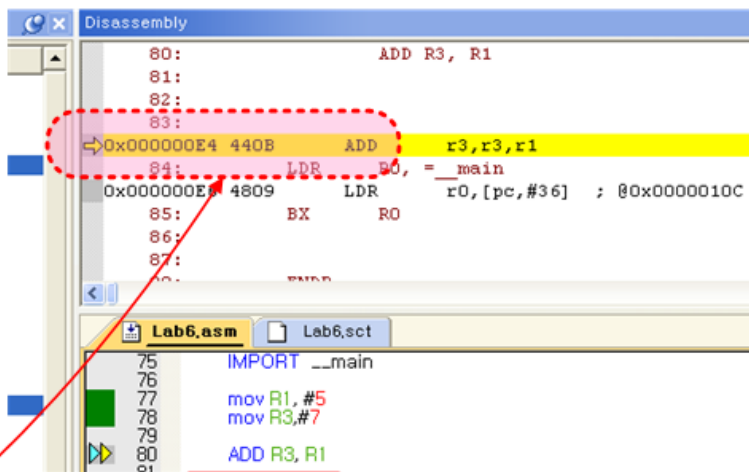
 Opcode



ADD R3, R1



0b0100_0100_0000_1011=0x440B



Hex2Bin provided by Sujin

Hexadecimal :

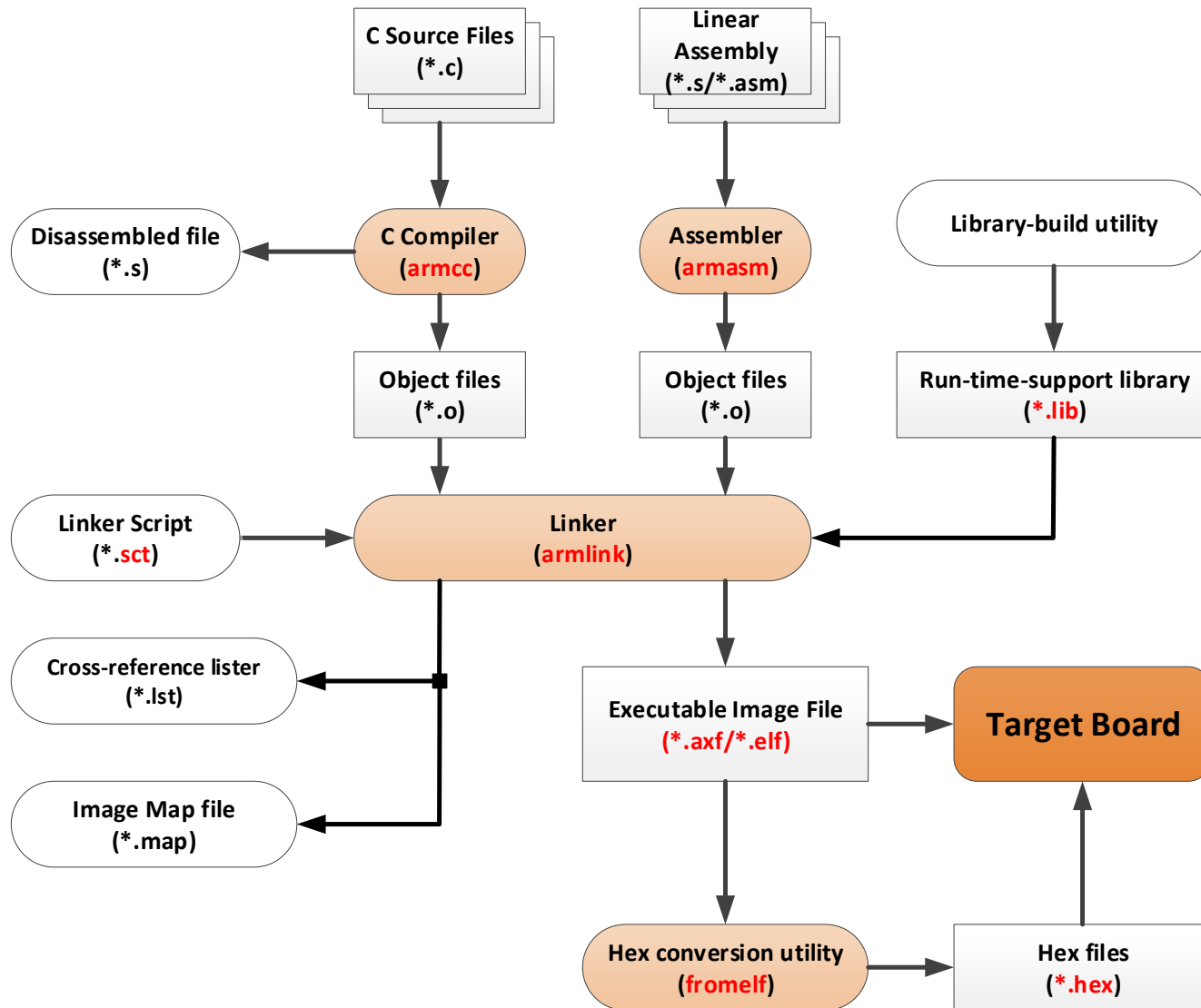
440B

>>
<<

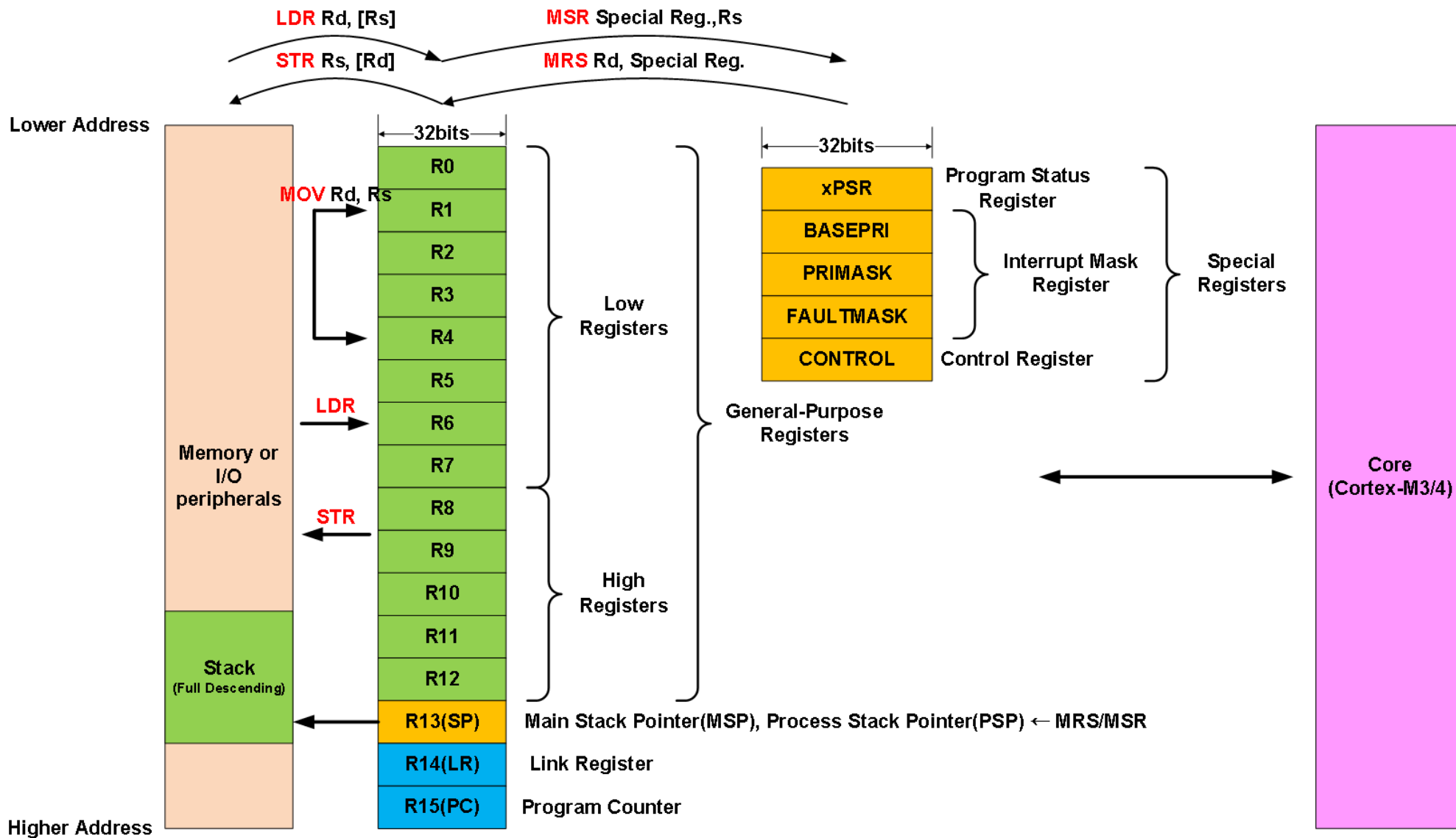
Binary :

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	0	0	0	0	1	0	1	1

KEIL MDK-ARM Toolchain

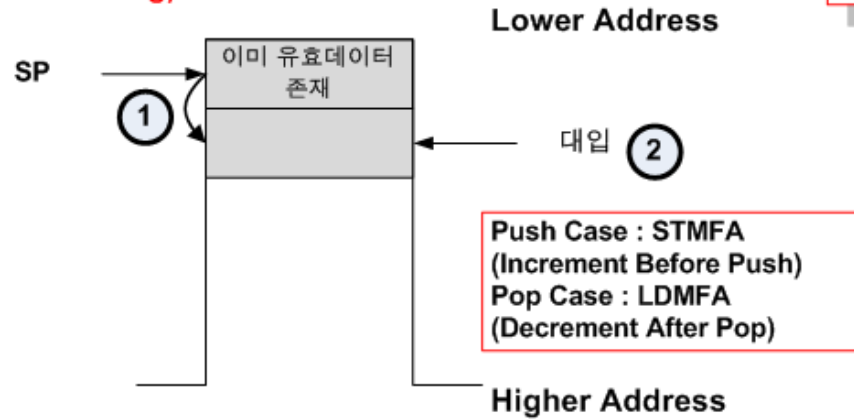


상수, 변수, 그리고, 함수 학습 - 1

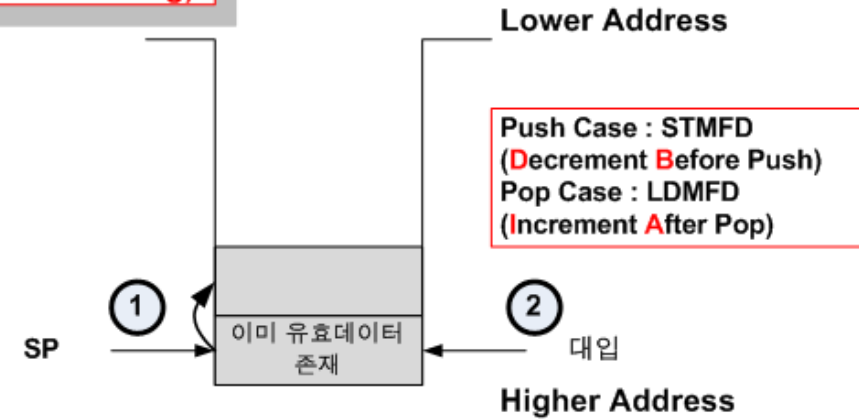


✓ Stack의 개념 :

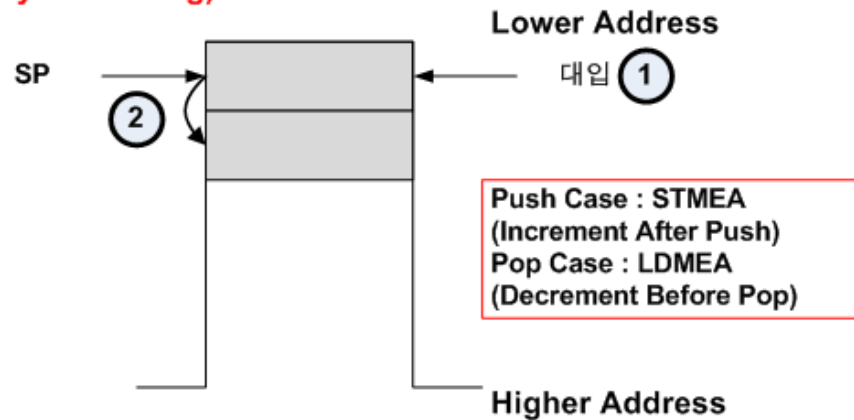
FA(Full Ascending)



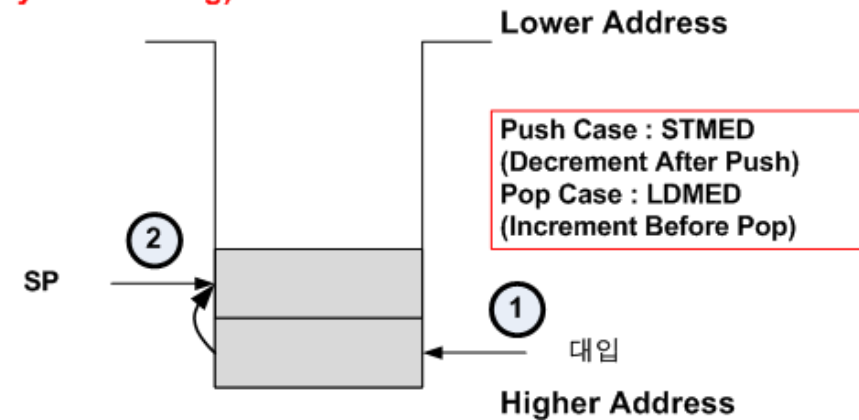
FD(Full Descending)



EA(Empty Ascending)



ED(Empty Descending)



✓ Heap의 개념 :

Heap	
관리 정보	
관리 정보	
관리 정보	

```
#include <stdlib.h>
```



```
자료형 *포인터_변수명;
```



```
포인터_변수명=(자료형 *)malloc(byte_단위의 기억 장소 크기);
```

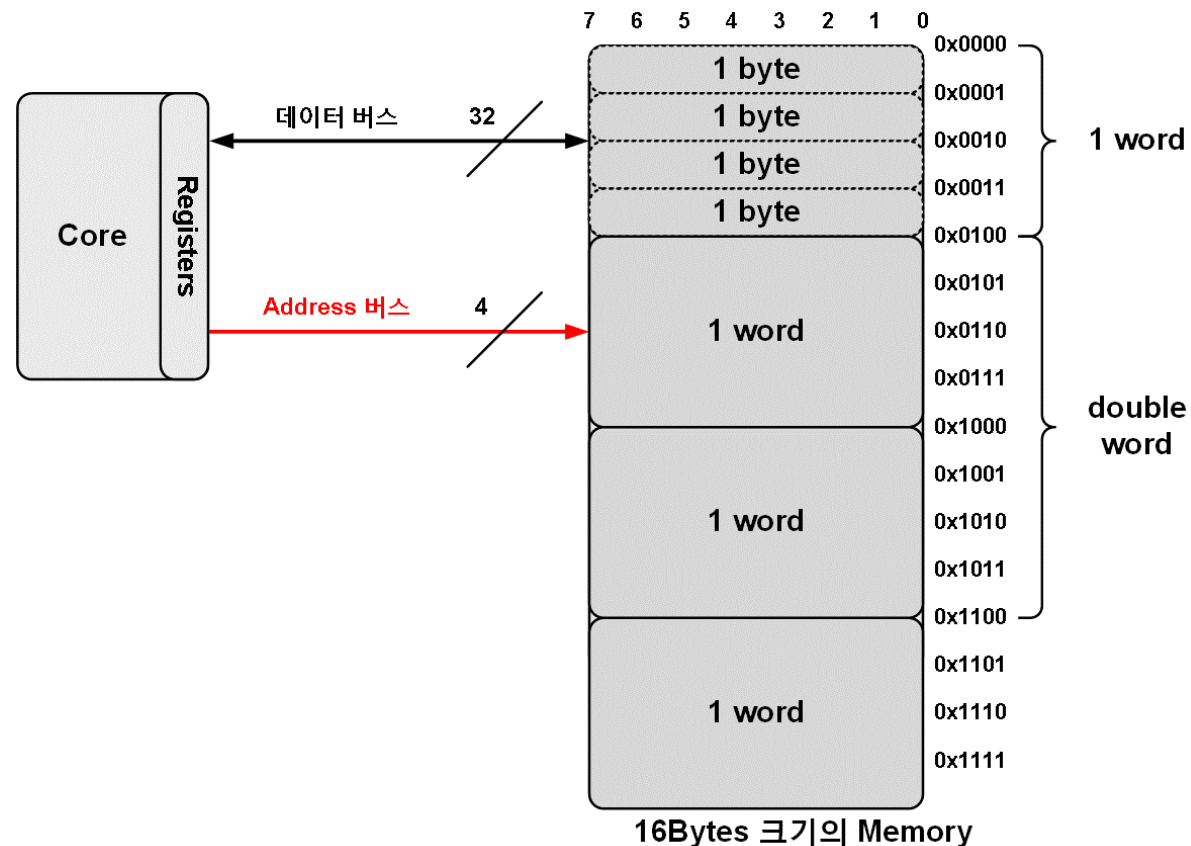


```
반복문을 이용하여 할당 받은 크기만큼 포인터_변수명에 할당;
```



```
free(포인터_변수명);
```

- ✓ Program : 명령어와 데이터의 순서 있는 모임.
- ✓ Bootloader : RESET interrupt가 발생하였을 때, 실행 이미지를 MCU 내부 flash memory에 저장해 주는 program.
- ✓ 번지(Address) : pointer 연산의 대상, 1byte 단위.



- ✓ 변수 : 데이터를 저장하는 공간. 함수 : 명령어와 데이터의 모임.
- ✓ Source file(*.c), Header file(*.h).

The screenshot shows a C source file named 'Ex2_1Src.c' in a code editor. The file contains several lines of comments (lines 1-17) and a function definition for 'main()' (lines 19-30). A red circle with the number '12' is placed over the 'Options for Target...' menu item. A bracket on the right side of the comments is labeled '주석(comments)'. An arrow points from a box labeled '함수 이름' (Function Name) to the 'int main()' line. Another bracket on the right side of the function body is labeled '함수의 내용' (Function Content).

```

1  /**
2
3  File Name : Ex1_3Src.c
4  Contents  : This example shows how to make C code.
5  Author   : Sujinpub
6  Date     : Nov. 2018
7
8  Revision History :
9      - V0.1, Nov. 2018: initial version.
10
11 All files for CodeMania provided by Sujinpub, including this file, are
12 provided as is without warranty of any kind, either expressed or implied.
13 The user assumes any and all risk from the use of this code.
14 It is the responsibility of the person integrating this code into an application
15 to ensure that the resulting application performs as required and is safe.
16
17 **/
18
19 int main()
20 {
21
22     char x=0, y=10, z;
23
24     // x variable is changed with 3.
25     x = 3;
26
27     z = x + y;
28
29 }
30

```

- ✓ C Code의 data type : ANSI C(ISO C89)의 기본적인 data types.

Data Types	전형적인 크기	전형적인 범위	동의어
char	1byte	-128~127	
short int	2bytes	-32,768~32,767	short
int	4bytes	$-2^{31} \sim (2^{31} - 1)$	long, long int
long long int	8bytes	$-2^{63} \sim (2^{63} - 1)$	
unsigned char	1byte	0~255	
unsigned short int	2bytes	0~65,535	unsigned short
unsigned long int	4bytes	$0 \sim (2^{32} - 1)$	unsigned, unsigned int, unsinged long
unsigned long long int	8bytes	$0 \sim (2^{64} - 1)$	

[표 2.1-1] 기본적인 C code data types 정리.

✓ C Code의 data type : ISO C99에서의 기본적인 data types.

▪ #include<stdint.h>, #include<stdbool.h>

Singed Types	전형적인 크기	전형적인 범위
<u>int8_t</u>	8bits	-128~127
<u>int16_t</u>	16bits	-32,768~32,767
<u>int32_t</u>	32bits	$-2^{31} \sim (2^{31} - 1)$
<u>int64_t</u>	64bits	$-2^{63} \sim (2^{63} - 1)$
<u>uint8_t</u>	8bits	0~255
<u>uint16_t</u>	16bits	0~65,535
<u>uint32_t</u>	32bits	$0 \sim (2^{32} - 1)$
<u>uint64_t</u>	64bits	$0 \sim (2^{64} - 1)$

[표 9.1.1-2] ISO C99에서의 기본적인 data types.

✓ C Code에서 함수의 개념 :

수학 시간에 학습한 함수(function)는 다음과 같이 입력 x 과 출력 y 사이의 관계로 표시했던 것을 기억할 수 있을 것입니다.

$$y = f(x)$$

동일하게 C code에서 함수는 다음과 같이 표시됩니다.

출력_매개변수의_data_type 함수의_이름(입력_매개변수_1 , 입력_매개변수_2,....)

✓ C Code에서 함수 :

- Visual C++의 빈프로젝트로 coding 및 debugging 실습.

✓ Google, Naver 등에서 Visual Studio 검색어로 찾는다.

- Visual Studio Community 2022를 설치.

Visual Studio 제품군을 만나보세요



Visual Studio
버전 17.1

Windows에서 .NET 및 C++ 개발자를 위한 최고의 포괄적인 IDE입니다. 소프트웨어 개발의 모든 단계를 향상시키고 개선할 수 있는 다양한 도구와 기능이 완벽하게 포함되어 있습니다.

Visual Studio 다운로드

- Community 2022
- Professional 2022
- Enterprise 2022



Visual Studio for Mac
버전 8.10

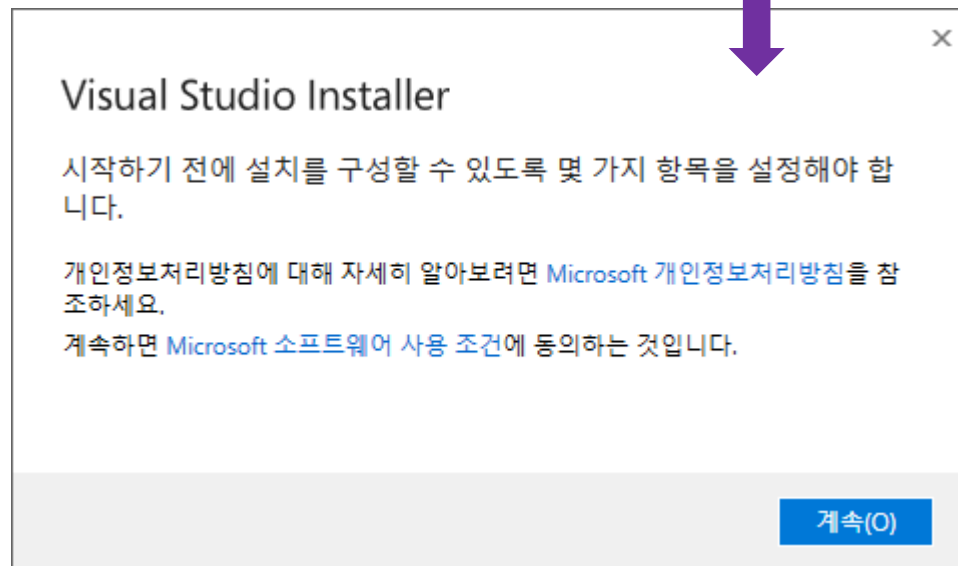
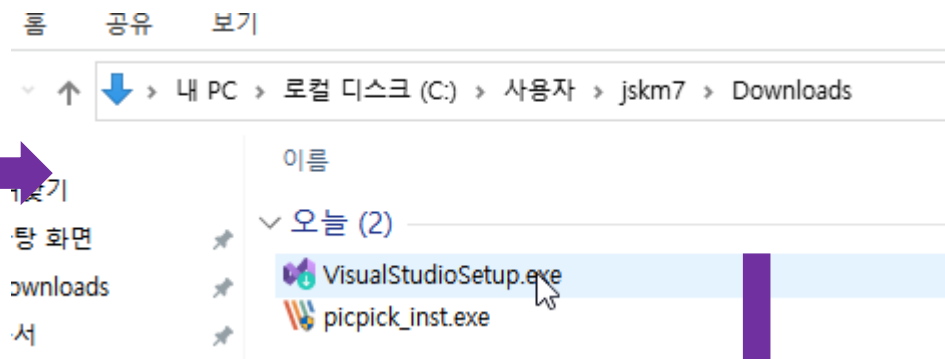
macOS에 기본 제공되는 .NET 개발자를 위한 포괄적인 IDE입니다. 웹, 클라우드 및 게임 개발에 대한 최 수준의 지원과 함께 플랫폼 간 모바일 앱을 만들기 위한 매우 유용한 도구가 포함되어 있습니다.

자세한 내용 읽기 [라이선스 활성화](#)

Visual Studio for Mac 다운로드

어떤 도구가 가장 적합한지 잘 모르시겠나요? 도움을 드릴 수 있습니다.

개발자 만나기



Visual Studio Installer

Visual Studio 설치 관리자를 가져옵니다.

다운로드 중: 6.77 MB/12.03 MB

1.56 MB

설치 중

취

설치 — Visual Studio Community 2022 — 17.1.0

워크로드 개별 구성 요소 언어 팩 설치 위치

ASP.NET 및 웹 개발
Docker 지원이 포함된 ASP.NET Core, ASP.NET, HTML/JavaScript 및 컨테이너를 사용하여 웹 애플리케이션을 빌드...

Azure 개발
.NET 및 .NET Framework를 사용하여 클라우드 앱을 개발하고 리소스를 만들기 위한 Azure SDK, 도구 및 프로젝트...

Python 개발
Python에 대한 편집, 디버깅, 대화형 개발 및 소스 제어입니다.

Node.js 개발
비동기 이벤트 구동 JavaScript 런타임인 Node.js를 사용하여 확장 가능한 네트워크 애플리케이션을 빌드합니다.

데스크톱 및 모바일 (5)

.NET을 사용한 모바일 개발
Xamarin을 사용하여 iOS, Android 또는 Windows용 플랫폼 간 애플리케이션을 빌드합니다. 여기에는 선택적 설치로 ...

.NET 데스크톱 개발
.NET 및 .NET Framework와 함께 C#, Visual Basic 및 F#를 사용하여 WPF, Windows Forms 및 콘솔 애플리케이션을...

C++를 사용한 데스크톱 개발 ☒
MSVC, Clang, CMake 또는 MSBuild 등 선택한 도구를 사용하여 Windows용 최신 C++ 앱을 빌드합니다.

유니버설 Windows 플랫폼 개발
C#, VB 또는 C++(선택 사항)를 사용하여 유니버설 Windows 플랫폼용 애플리케이션을 만듭니다.

위치

C:\Program Files\Microsoft Visual Studio\2022\Community 변경...

계속하면 선택한 Visual Studio 버전에 대한 라이선스에 동의하게 됩니다. Microsoft는 Visual Studio와 함께 다른 소프트웨어를 다운로드할 수 있는 기능도 제공합니다. 이 소프트웨어는 [타사 고지 사항](#) 또는 해당 라이선스에 명시된 것처럼 별도로 라이선스가 부여됩니다. 계속하면 이러한 라이선스에도 동의하게 됩니다.

설치 세부 정보

Visual Studio 핵심 편집기

C++를 사용한 데스크톱 개발

포함됨

C++ 핵심 데스크톱 기능

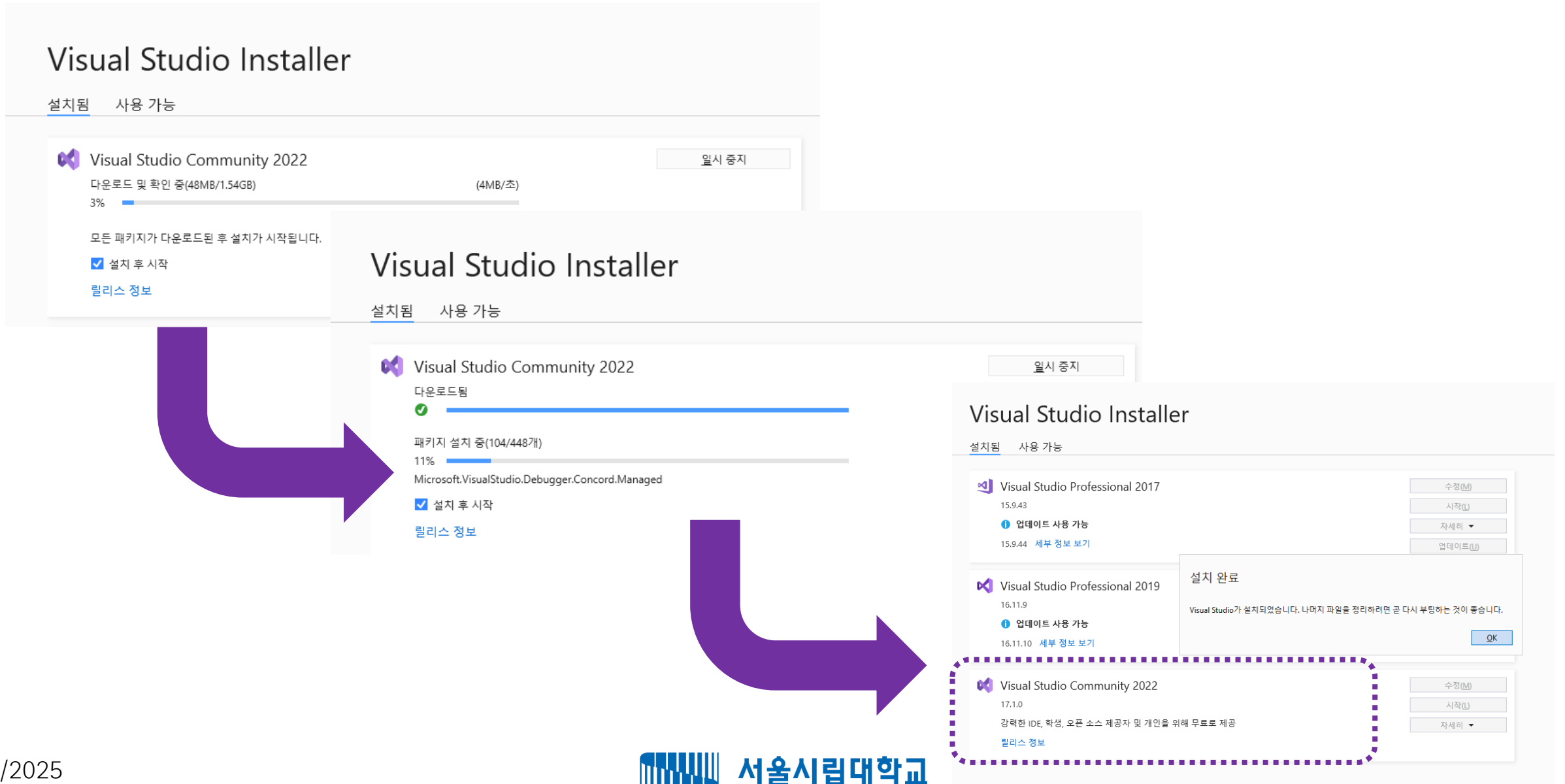
선택 사항

- ☒ MSVC v143 - VS 2022 C++ x64/x86 빌드...
- ☒ Windows 10 SDK(10.0.19041.0)
- ☒ Just-In-Time 디버거
- ☒ C++ 프로파일링 도구
- ☒ Windows용 C++ CMake 도구
- ☒ 최신 v143 빌드 도구용 C++ ATL(x86 및 x64)
- ☒ Test Adapter for Boost.Test
- ☒ Test Adapter for Google Test
- ☒ Live Share
- ☒ IntelliCode
- ☒ C++ AddressSanitizer
- ☐ MSVC v143 - VS 2022 C++ ARM64 빌드 도...
- ☐ 최신 v143 빌드 도구용 C++ MFC(x86 및 x6...
- ☐ v143 빌드 도구에 대한 C++/CLI 지원(최신)
- ☐ v143 빌드 도구용 C++ 모듈(x86 및 x64)...

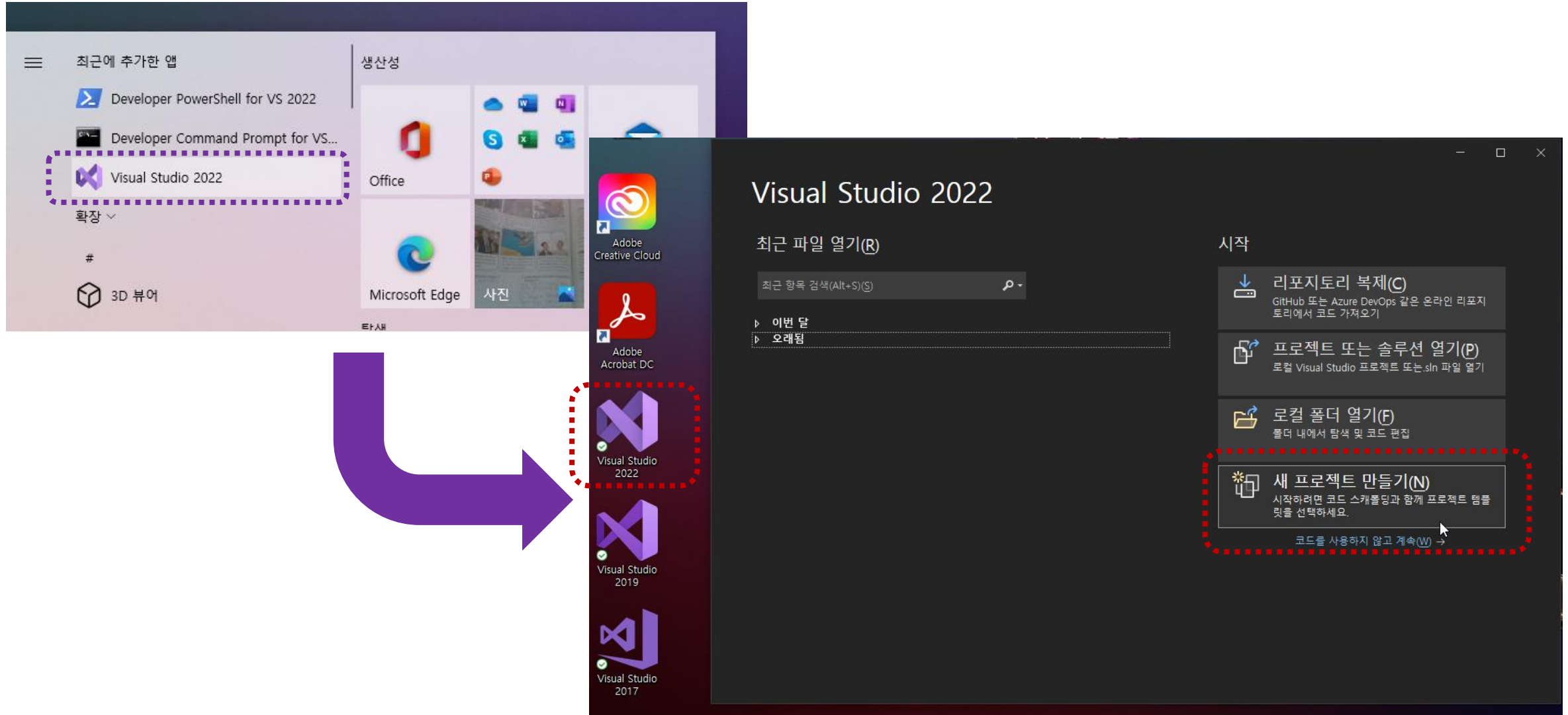
필요한 전체 공간 5.74GB

모두 다운로드한 후 설치

설치



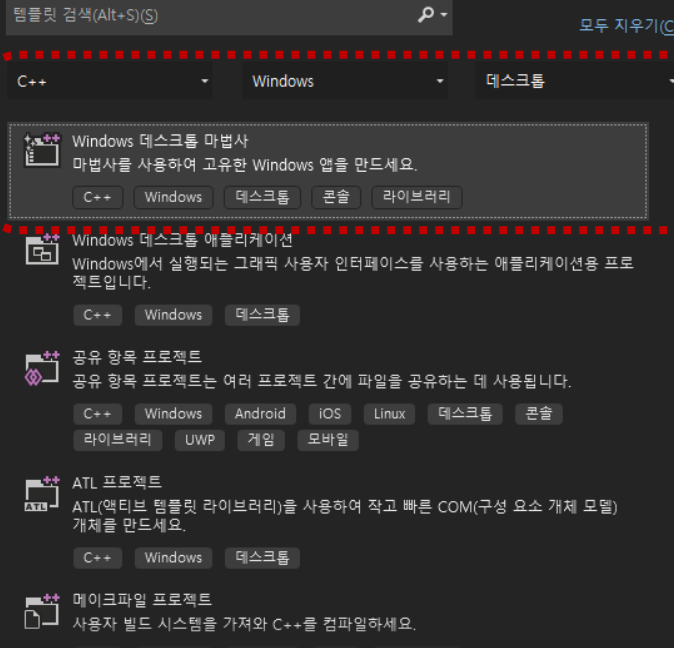
Visual Studio Community 설치 방법 - 4



새 프로젝트 만들기

최근 프로젝트 템플릿(R)

최근에 액세스한 템플릿 목록이 여기에 표시됩니다.



뒤로(B) 다음(N)

새 프로젝트 구성

Windows 데스크톱 마법사

프로젝트 이름(I)

Project1

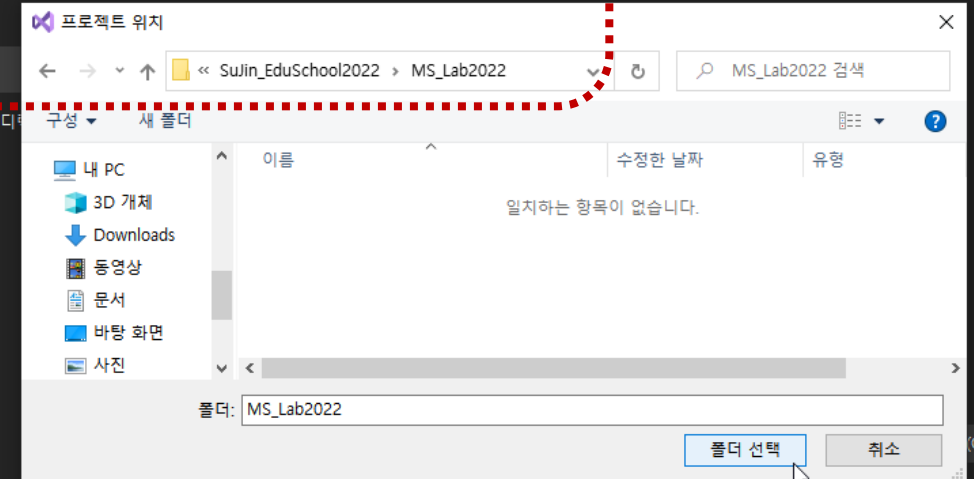
위치(L)

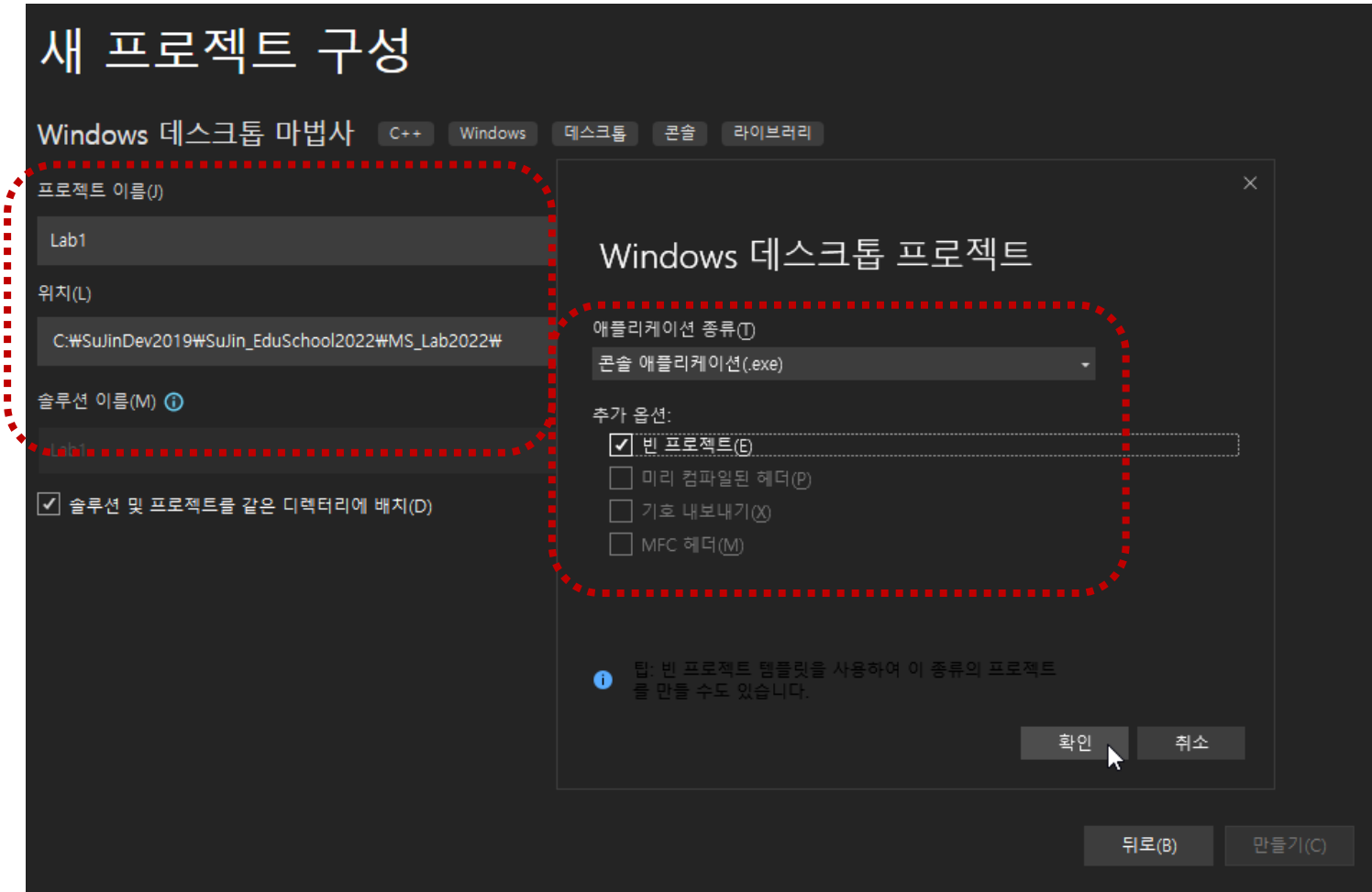
C:\Users\Wjskm7\source\repos

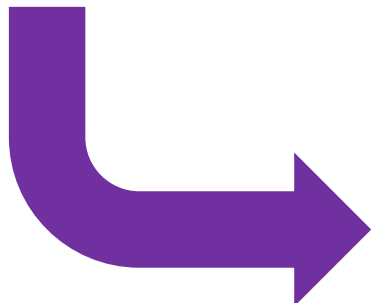
솔루션 이름(M)

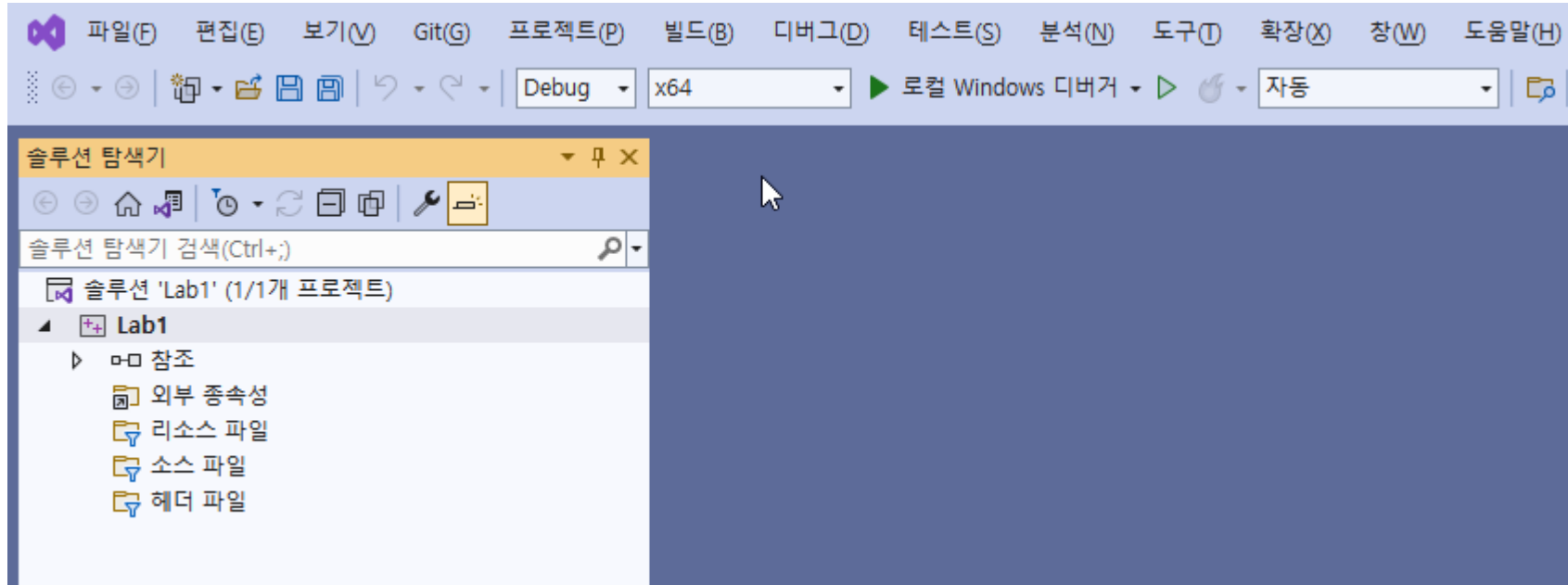
Project1

☐ 솔루션 및 프로젝트를 같은 디렉토리에 배치









C 언어에서 함수 실습.

✓ 복사 생성자 :

The screenshot shows the Visual Studio Code interface with a C program open in the editor. The file explorer on the left shows the project structure, including 'Lab1.c'. The editor displays the following code:

```

1  #include<stdio.h>
2  #include<stdint.h>
3
4  int32_t Sum(int32_t x, int32_t y);
5
6  int main() {
7
8      int32_t a = 0;
9
10     a = Sum(2, 3);
11
12     return 0;
13 }
14
15 int32_t Sum(int32_t x, int32_t y) {
16     int32_t z = 0;
17
18     z = x + y;
19
20     return(x + y);
21 }
    
```

The code defines a function `Sum` that takes two `int32_t` arguments and returns their sum. The `main` function calls `Sum(2, 3)` and stores the result in `a`.

```

Ch2Ex1.c x
1  /*****
2  * Project: Ch2Ex1
3  * Function:
4  *****/
5  * Author: Dr. JongSu Lim(Fermi Lim)
6  * Version: 1.0
7  *****/
8  *
9  * Copyright 2015-12-30 by SuJin Electronics Inc.
10 *
11 *****/
12
13 #include<stdio.h>
14
15 int main()
16 {
17     int x=0x12345678;
18     unsigned char *px=(unsigned char *)&x;
19
20     if(*px == 0x78) { // 최하위 byte 비교!
21         printf("This computer is based of Little Endian\n");
22     } else {
23         printf("This computer is based of Big Endian\n");
24     }
25
26     return 1; // main() 함수는 int data type return 요구!
27 }
28

```

함수 형식(prototype) 선언
출력 매개변수 함수이름(입력 매개변수들)

사용할 변수 선언/정의

함수 몸체
데이터를 이용한 구체적인 명령관련 내용

함수의 return 처리.
(필요시 return문 사용)

BML_1

함수에 대한 학습 - 2.

출력_매개변수_data_type 함수_이름(입력_매개변수__data_type 입력_매개변수,);

함수 형식(prototype) 선언

```
main()
{
    main() 함수에서 필요한 변수들 선언 및 정의;
    출력 매개변수와 입력 매개변수 선언 및 정의;

    ... 중간 생략 ...
```

출력_매개변수=함수_이름(입력_매개변수);

함수 호출

```
    ... 중간 생략 ...
}
```

출력_매개변수_data_type 함수_이름(입력_매개변수__data_type 입력_매개변수,)

함수 header

```
{
    함수에서 필요한 변수들 선언 및 정의;
```

... 함수 내용 ...

함수 body

함수 정의

BML_2

✓ #define 문을 이용한 Macro :

- #define 전처리 지시어는 지정한 기호(identifier)를 정의한 문자열로 바꾸어 주는 역할을 수행

치환

#define identifier 정의한_문자열

```
#include <stdio.h>
#define MyName "Lim JongSu"
int main() {
    printf(MyName);
    return 1;
}
```

치환

#define identifier(입력_매개변수_list)

입력_매개변수_list를
포함하는 문자열들

✓ #define 문을 이용한 Macro :

- macro는 함수가 아니다

```
#include <stdio.h>
#define MyMean(a,b) (a+b)/2

int main() {
    float x=7.0f, y=8.0f, z;
    z=MyMean(x,y);

    return 1;
}
```

```
#include <stdio.h>
#define MyEquation(a) a*a+a+1

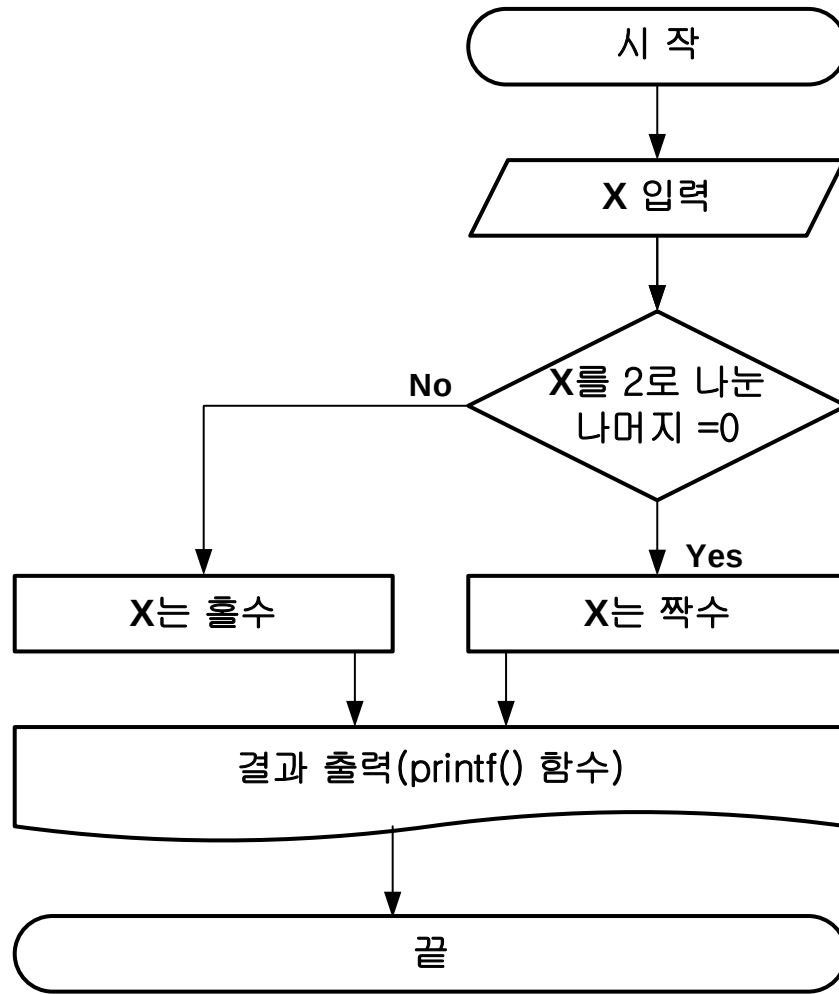
int main() {
    float x=7.0f, y, z;

    y=MyEquation(x);
    z=MyEquation(++x);
    return 1;
}
```

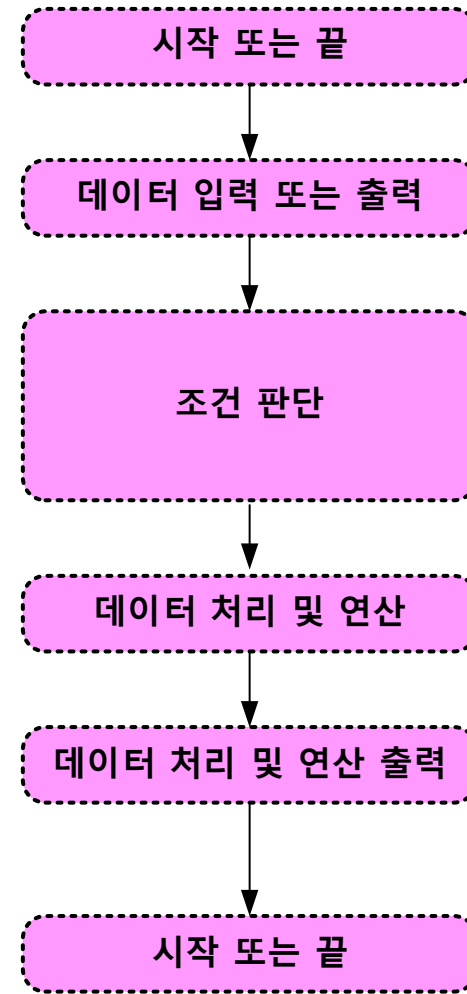
```
#include <stdio.h>
#define Num      10
#if (Num > 1)    // #if 조건식
#define MyMean(a, b) (a+b)/2
#endif

int main() {
    float x1=7.0f, x2=8.0f, y;
    y=MyMean(x1,x2);
    return 1;
}
```

흐름제어에 대한 학습 - 1.



(a) 순서도(flowchart).



(b) 처리 절차.

BML_3

전형적인 사용방법(Syntax) : if~else문

a) if (수식)

명령문1; // 수식이 참인 경우.

else

명령문2; // 수식이 거짓인 경우.

단, 수식이 참인 경우만 판단하고 싶다면, else 문은 없어도 된다.

Ch2Ex2

전형적인 사용방법(Syntax) : **switch~case문**

```
a) switch (상수 수식)    // 상수 : 정수 상수, 문자 상수
    case (상수1) : // 상수 수식의 결과가 상수1인 경우.
    {
        ... 명령문 나열 ...
        break;
    }
... case문 나열 ...
default : // 상수 수식의 결과와 일치하는 case문이 없는 경우.
{
    ... 명령문 나열 ...
    break;
}
```

Ch2Ex3

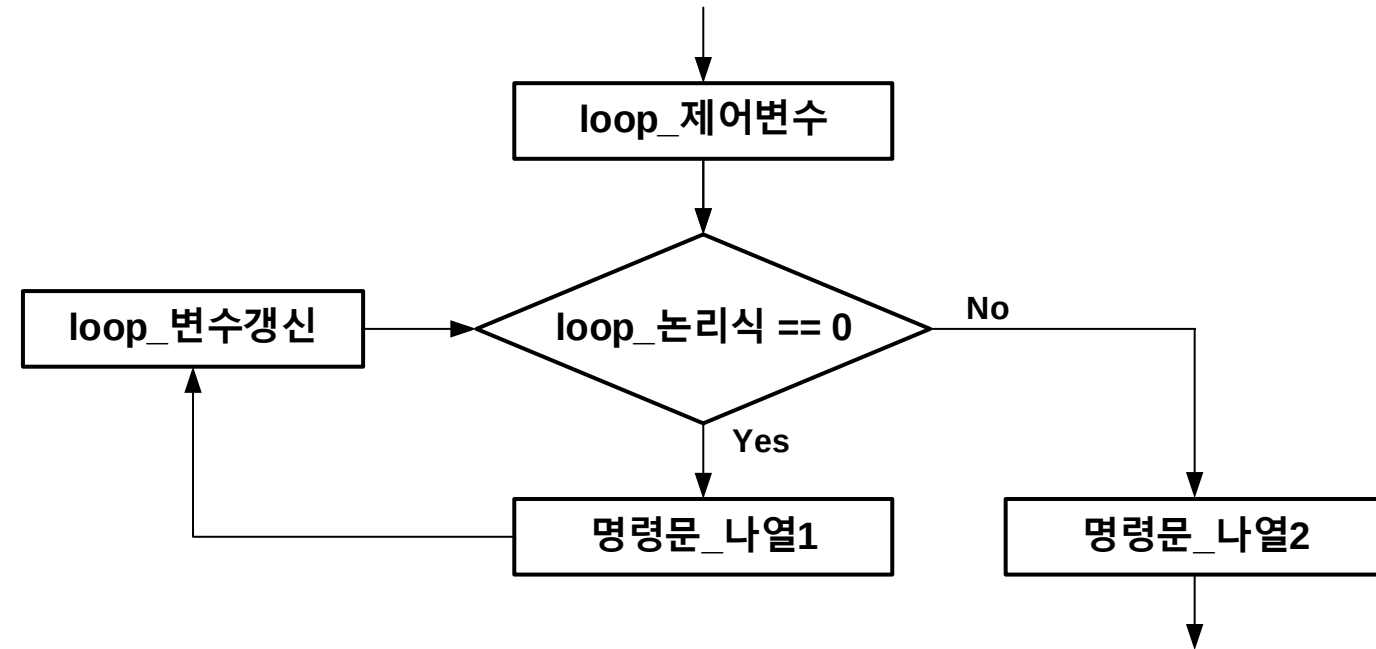
단, case문의 개수에는 제한이 없다.

흐름제어에 대한 학습 - 4.

```

    1 for (loop_제어변수; loop_논리식; loop_변수갱신) {
        2     ... 명령문 나열1 ...
        3     ...
    }
    4     ... 명령문 나열2 ...
    3
  
```

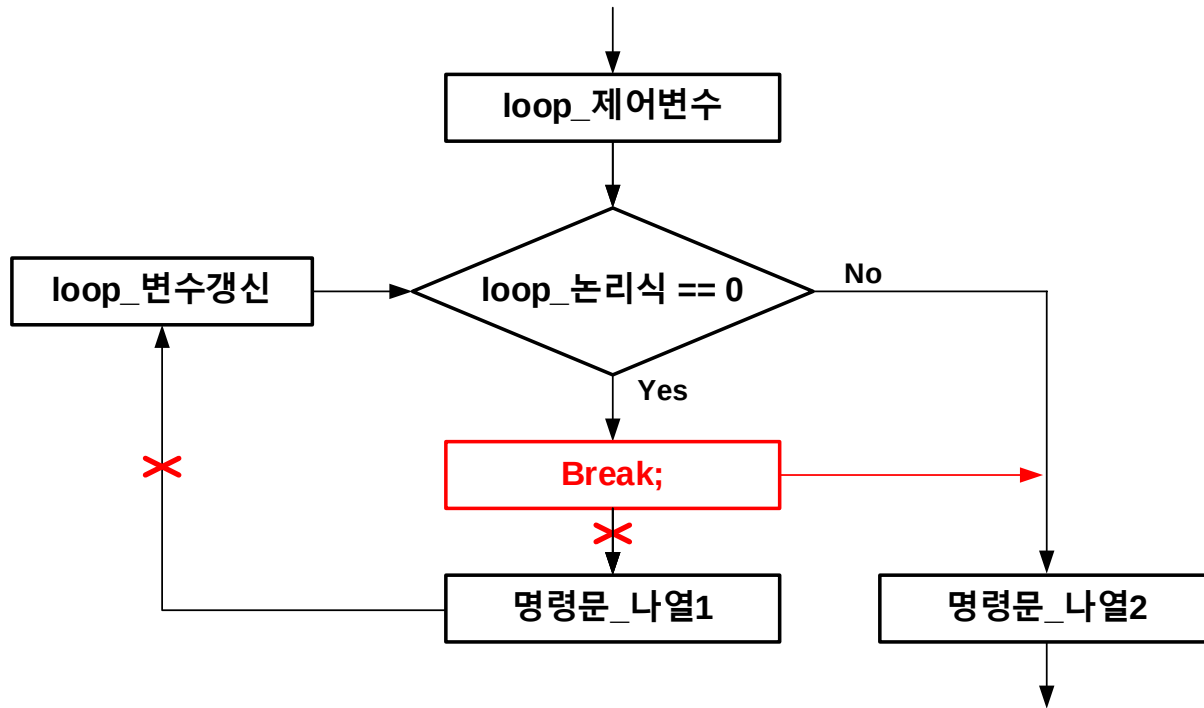
(a) for loop 실행 순서.



BML_4

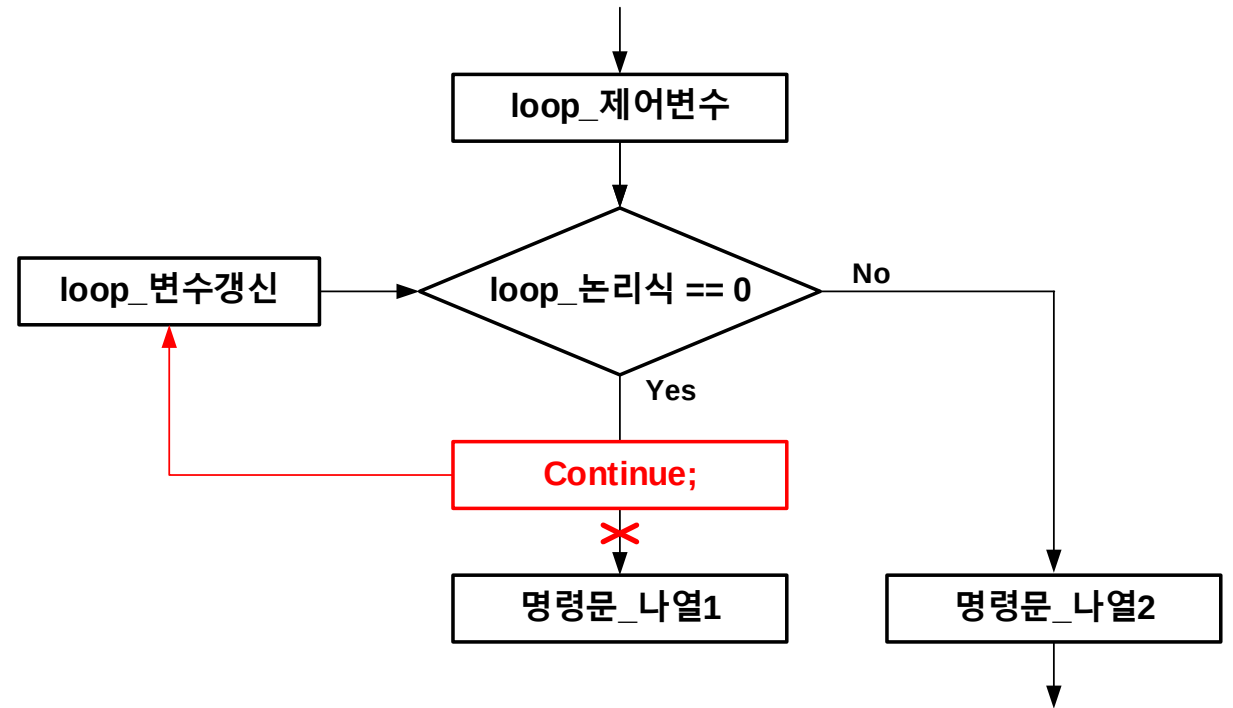
(b) for loop 실행 순서(flow chart)

Ch2Ex4



(a) 반복문에서 **break**문의 경우.

BML_5



(b) 반복문에서 **continue**문의 경우.

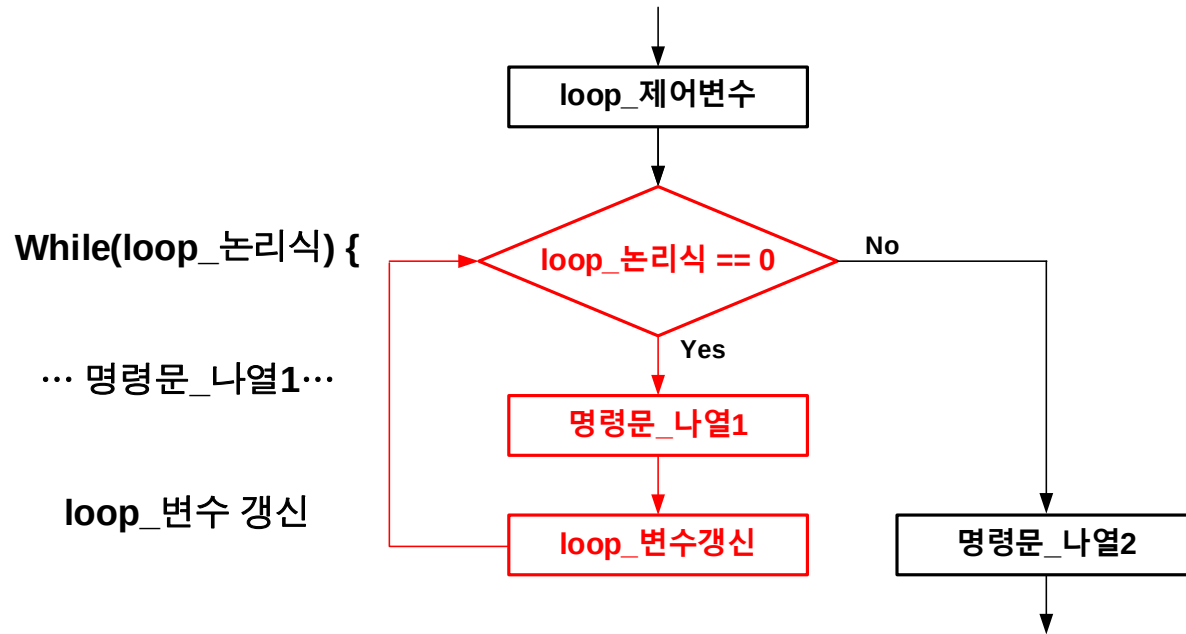
전형적인 사용방법(Syntax) : **while loop**

```
a) while (loop_논리식) {  
    ... 명령문 나열1 ...  
}  
... 명령문 나열2 ...
```

전형적인 사용방법(Syntax) : **do~while loop**

```
a) do {  
    ... 명령문 나열1 ...  
} while (loop_논리식);  
... 명령문 나열2 ...
```

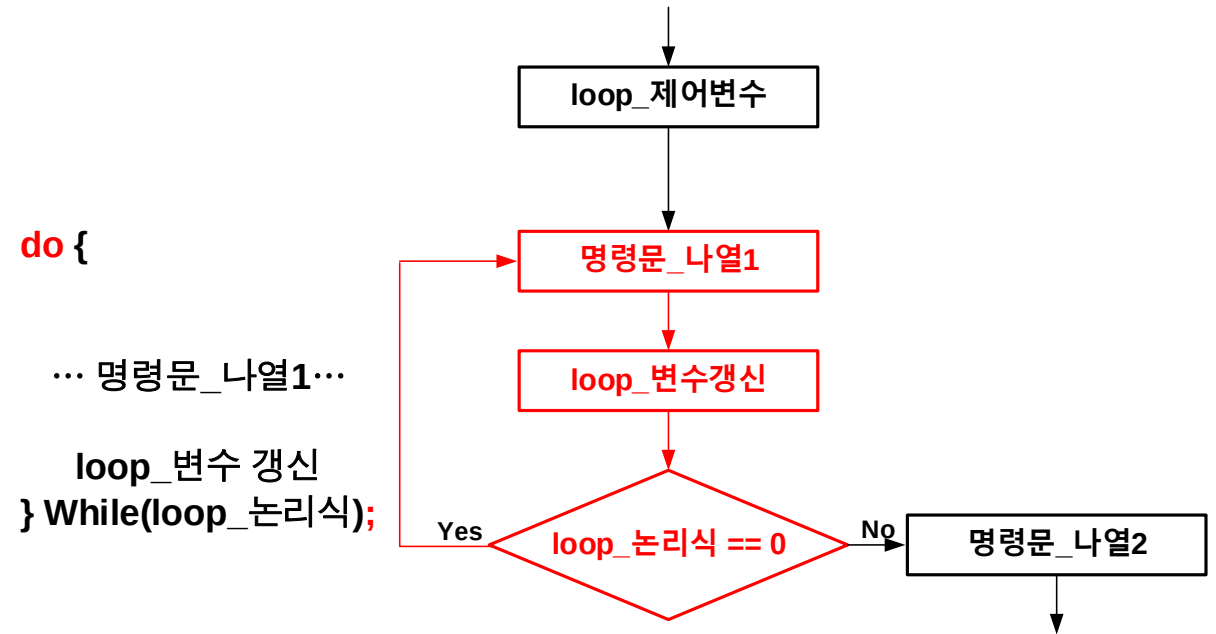
흐름제어에 대한 학습 - 7.



(a) while loop문의 경우.

BML_7

Ch2Ex4를 while loop로 수정!



(b) do~while loop문의 경우.

Ch2Ex4를 do-while loop로 수정!

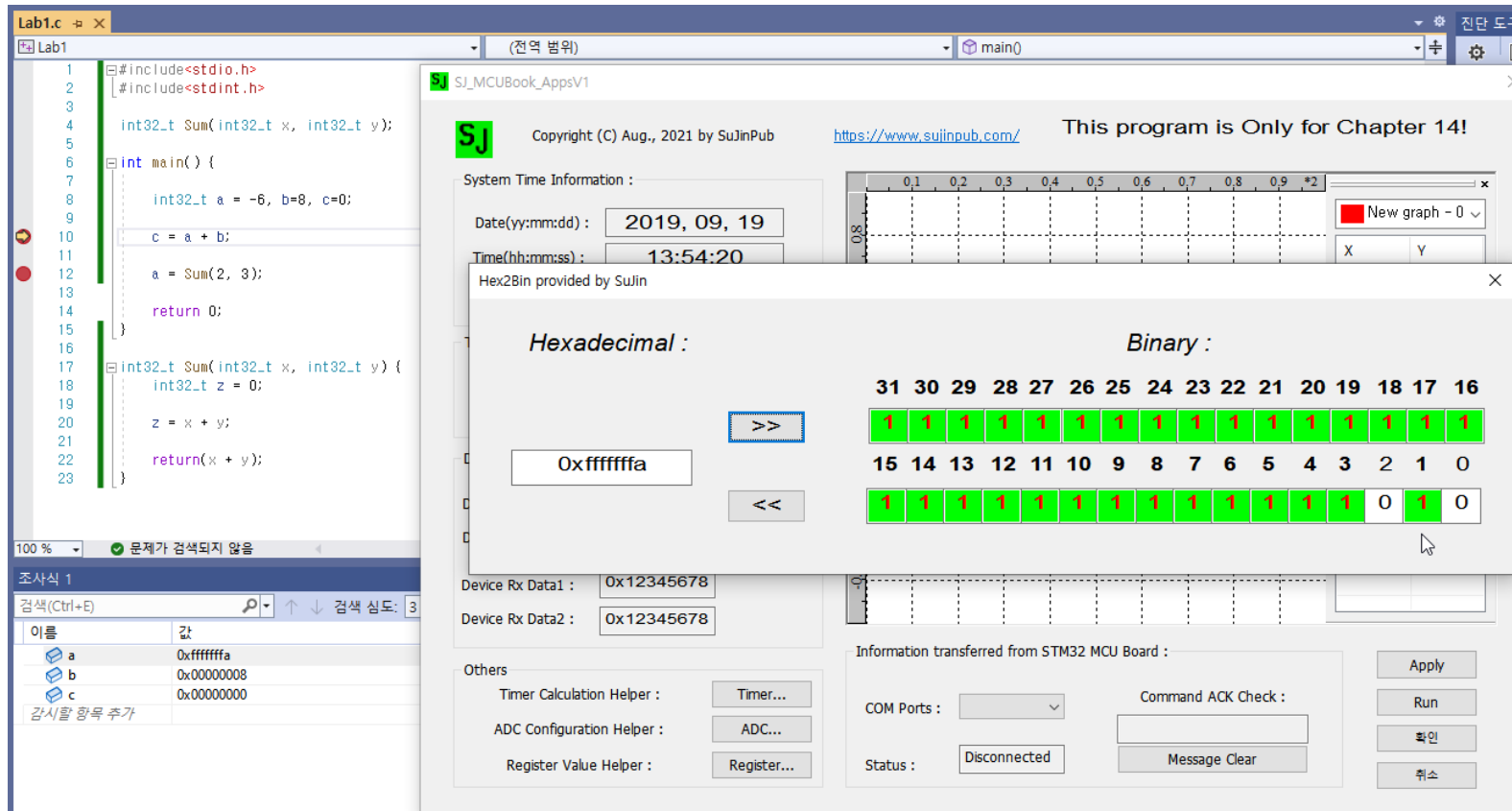
연산자 우선 순위.

연산자	설 명	우선 순위	연산자	설 명	우선 순위
변수, 상수		16	+, -	덧셈, 뺄셈	12
a[k]	배열의 첨자		<<, >>	shift 연산	11
f()	함수호출		<, >, <=, >=	관계 연산	10
.	구조체의 “.”		==, !=		9
->	구조체의 “->”	15	&	bit 연산, and	8
++, --	“1”씩 증감		^	bit 연산, xor	7
sizeof	byte 단위 크기			bit 연산, or	6
~	bit 연산, not		&&	논리, and	5
!	논리 연산, not			논리, or	4
-, +	음수, 양수 부호		?:	조건 연산	3
&	pointer 관련		=, +=, -=, *=, /=, %=, <<=, >>=, =, &=, ^=, =, ~=	대입 연산자 (산술, shift 연산, bit 연산)	2
*	pointer 관련				
(type)	cast 연산자	14			
*, /, %	곱셈, 나눗셈, 나머지	13	,	comma 연산	1

BML 8

산술 연산자	기 능	사용 방법
+	덧셈	<code>int x=7 + 3; // x=10</code>
-	뺄셈(2의 보수 연산)	<code>int x=7 - 3; // x=4</code>
*	곱셈	<code>int x=7 * 3; // x=21</code>
/	나눗셈	<code>double x=7 / 3.0; // x=2.3</code>
%	나머지를 구해줌	<code>int x=7 % 3; // x=1</code>

- ✓ 2의 보수와 1의 보수 개념 :
 - 모든 Computer 계산은 2의 보수로 수행.



논리와 비트 연산자

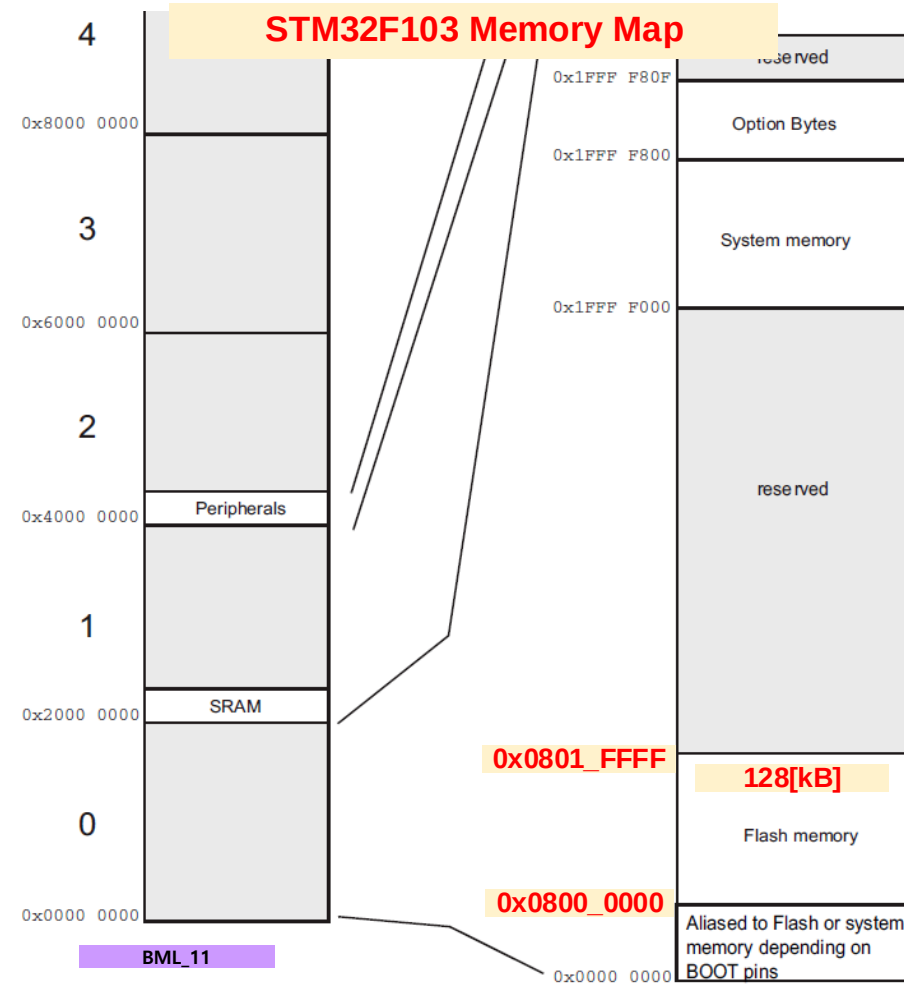
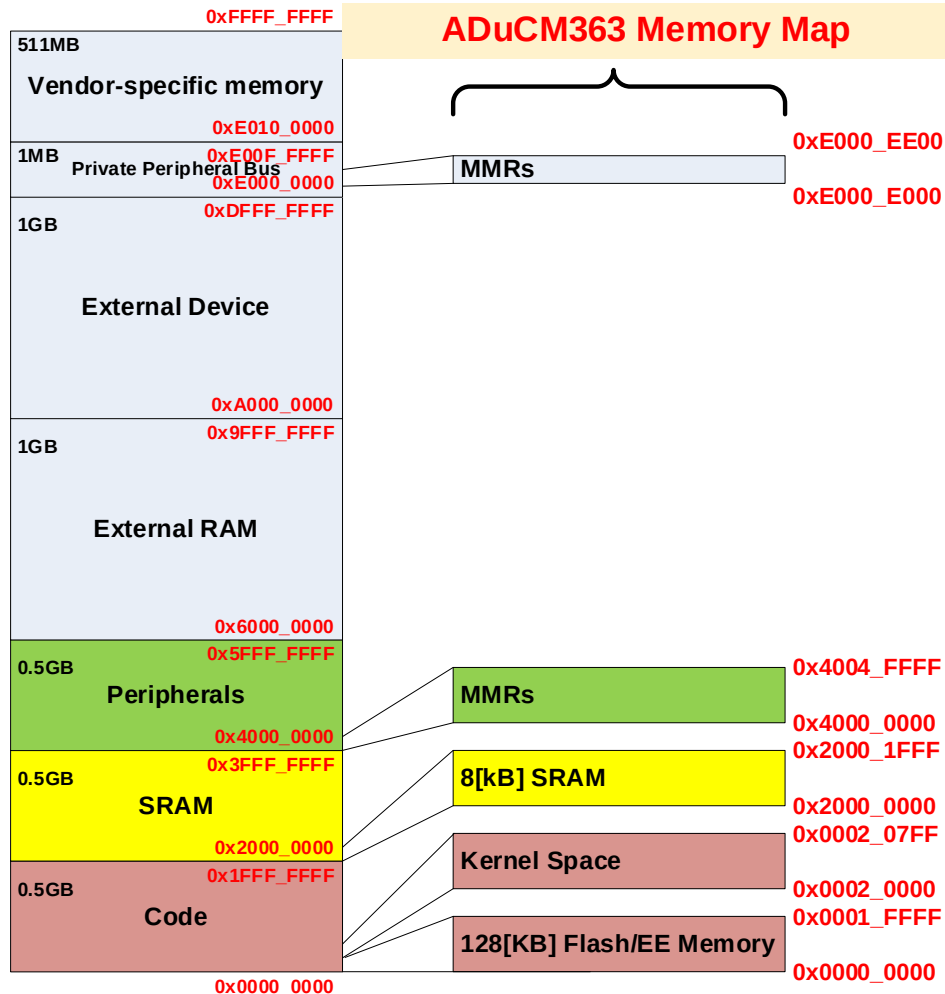
연산자	논리 연산자	Bit 연산자
AND	&&	&
OR		
XOR	unsupported	^
NOT	!	~
	조건 판단문에서 사용	bit-field 제어에 사용



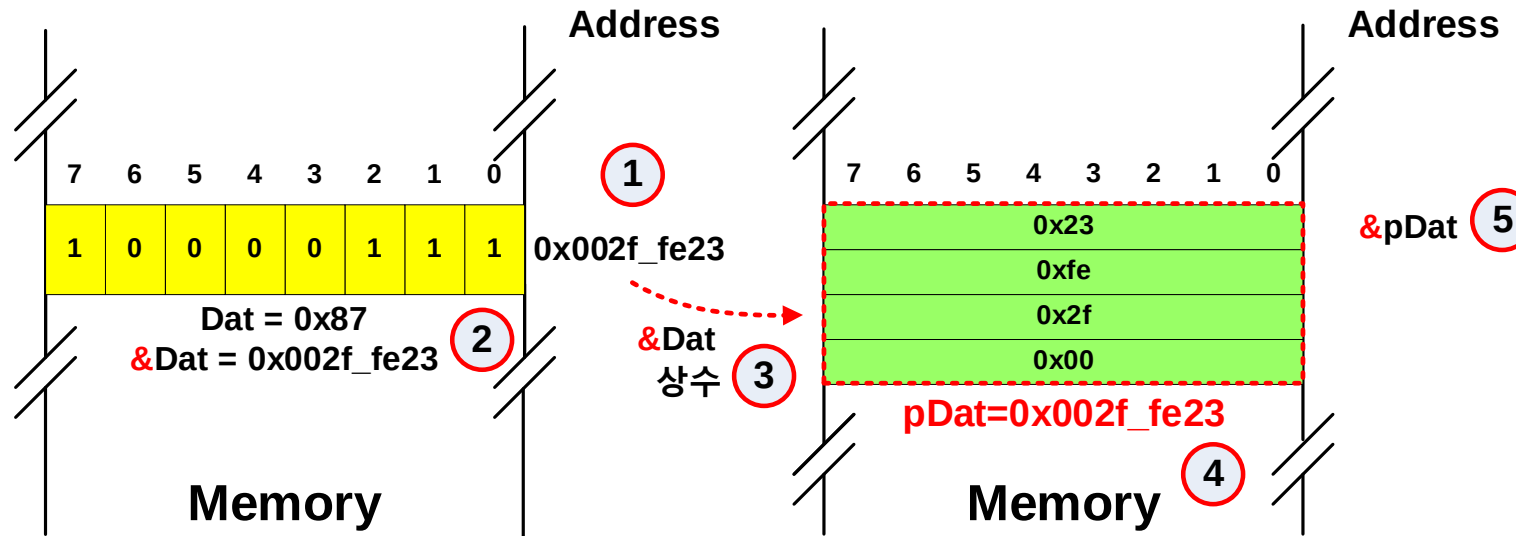
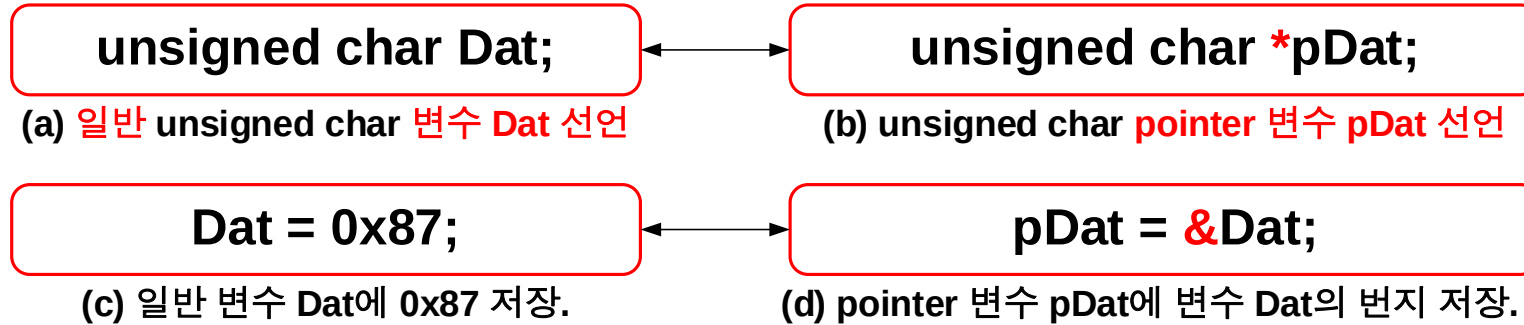
BML_9

연산자	기능 설명
$x \ll y$	변수 x 에 저장되어 있는 값을 구성하는 bits를 y 만큼 왼쪽으로 이동. 단, 왼쪽으로 밀려난 bits는 제거되고, 오른쪽에 빈 bits에는 “0”을 채움.
$x \gg y$	변수 x 에 저장되어 있는 값을 구성하는 bits를 y 만큼 오른쪽으로 이동. 단, 오른쪽으로 밀려난 bits는 제거, 왼쪽에 빈 bits에는 부호 bit 또는 “0”으로 채움.

✓ Cortex-M memory map과 STM32F103과 ADuCM363 memory map :



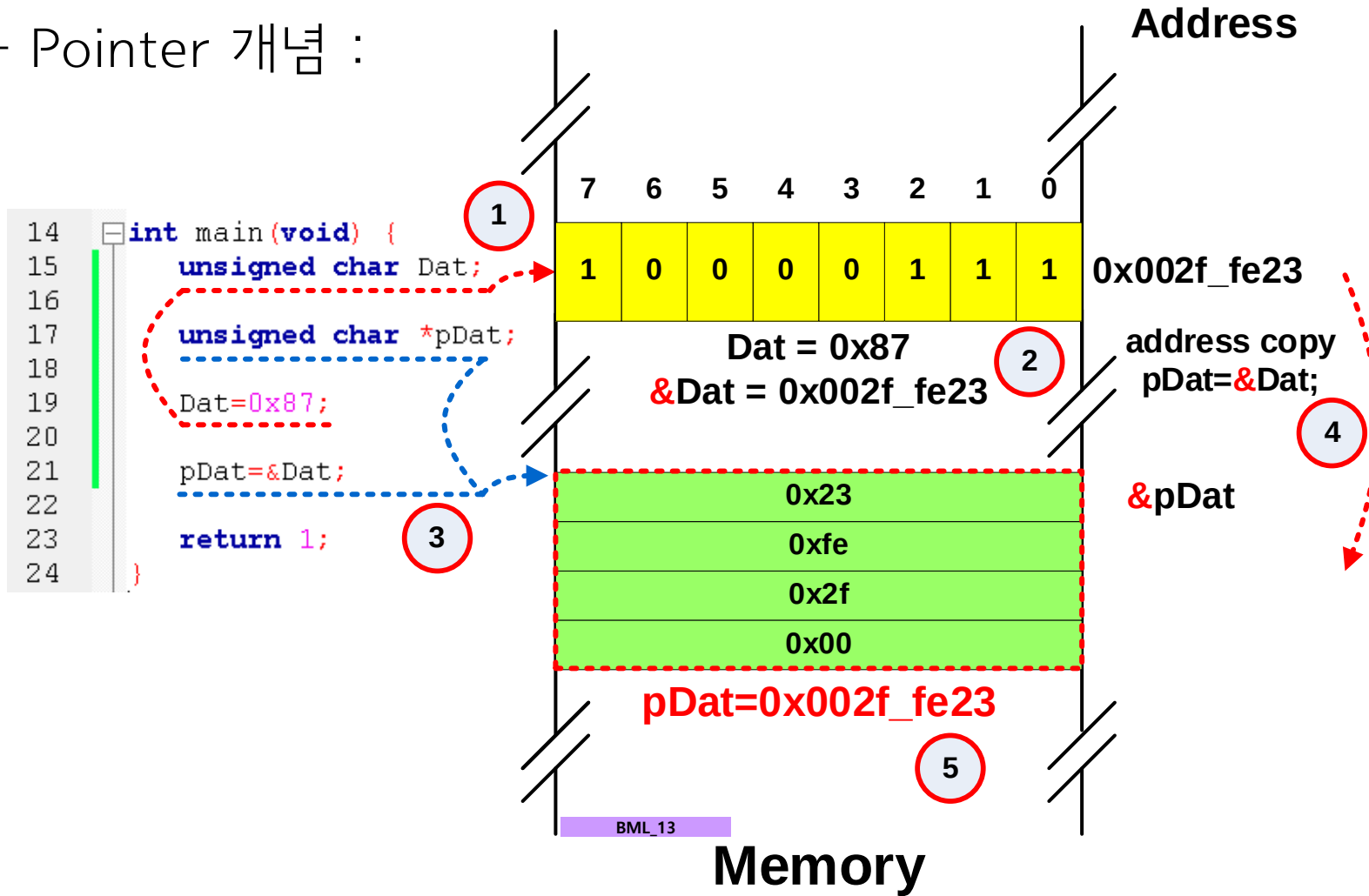
✓ C/C++ Pointer 개념 :



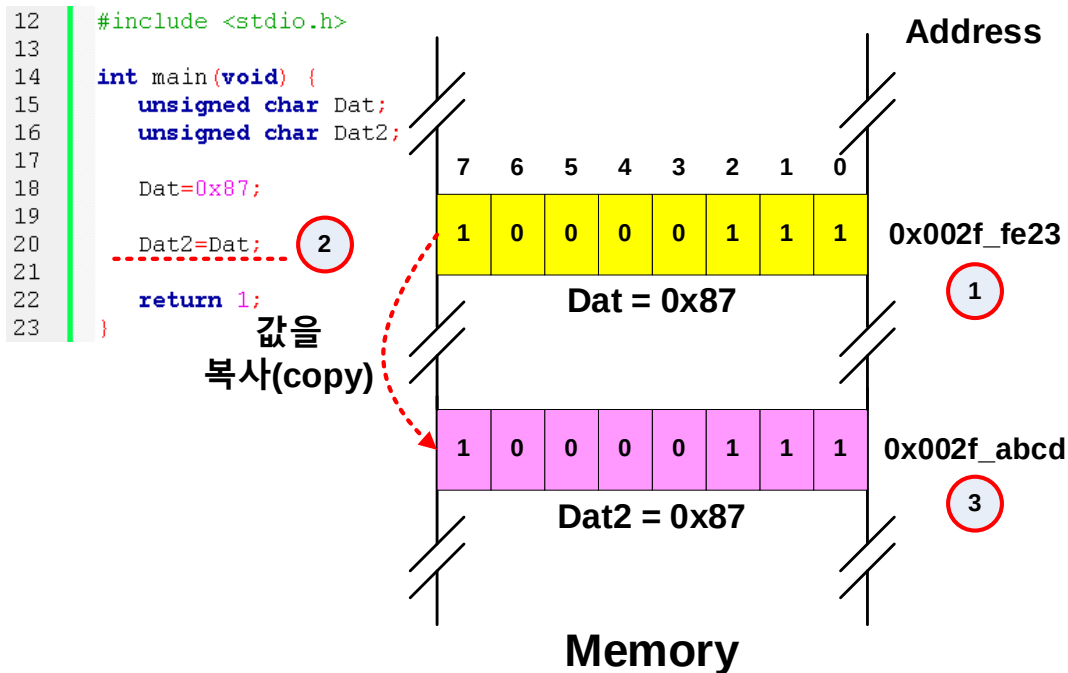
BML_12

변수와 pointer에 대한 학습 - 3.

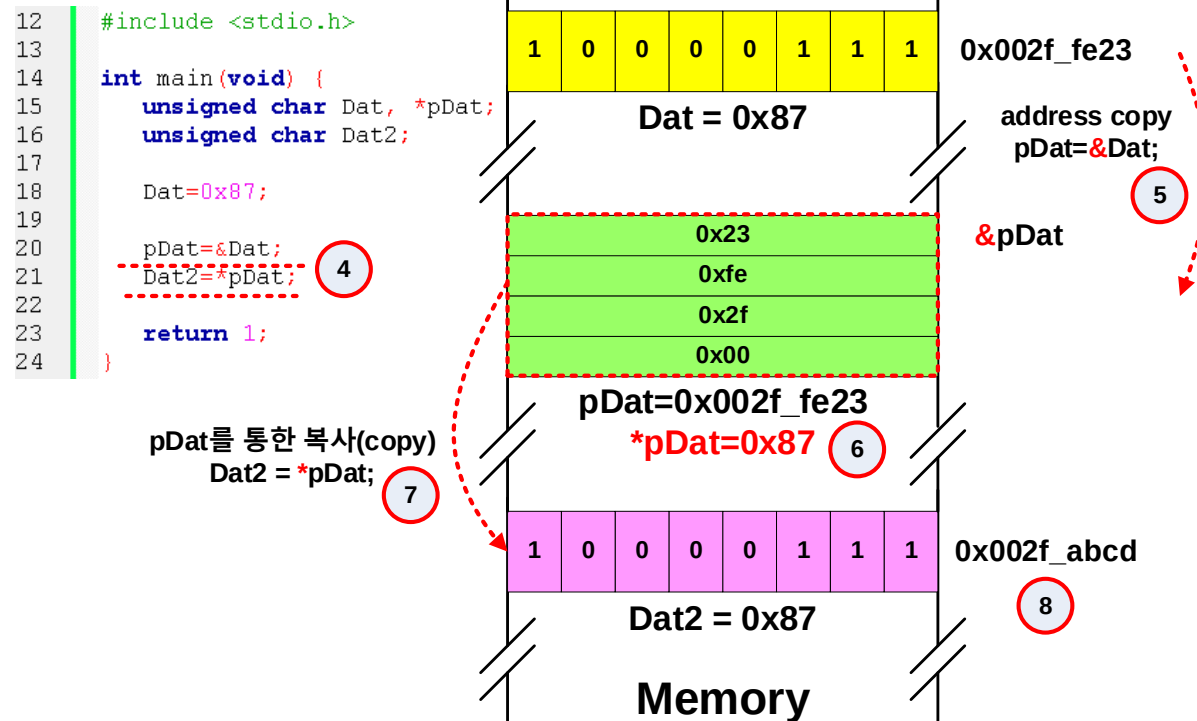
✓ C/C++ Pointer 개념 :



✓ C/C++ Pointer 개념 :



(a) 변수 사이의 **직접** 값 복사.



(b) address를 통한 **간접** 값 복사.

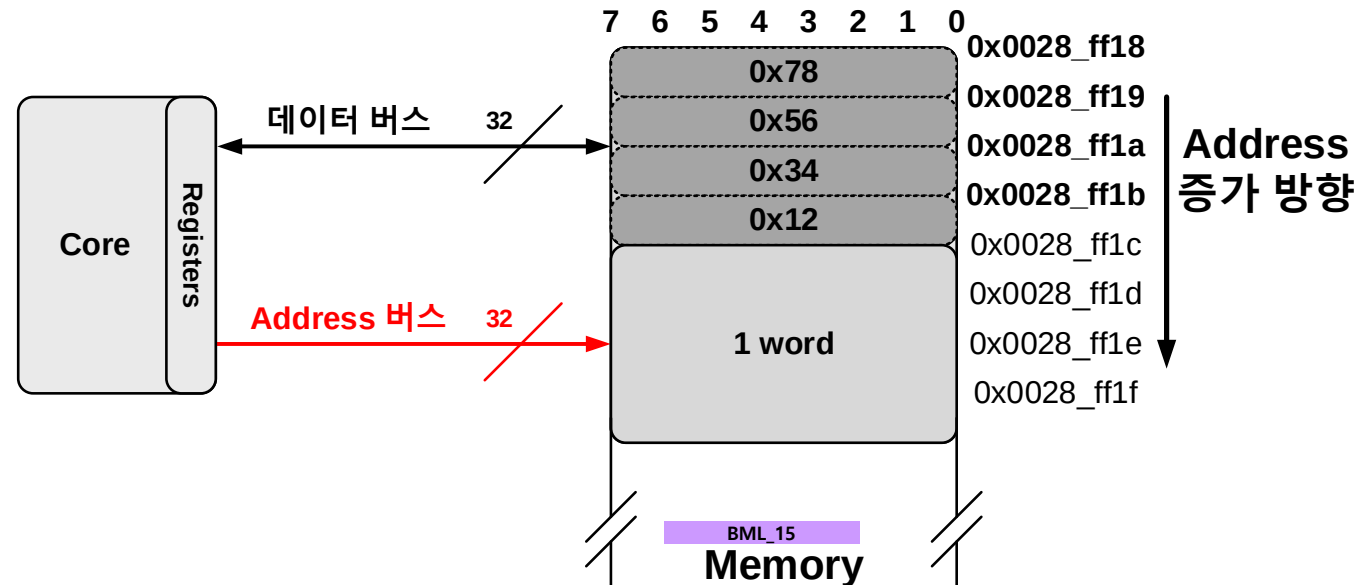
BML_14

변수와 pointer에 대한 실습.

- ✓ C/C++ Pointer 개념 :
- 1) Ch2_5.c

```

Ch2Ex1.c [Ch2Ex1] - Code::Blocks 13.12
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
Ch2Ex1.c
6  * Version: 1.0
7  *****
8  *
9  * Copyright 2015-12-30 by SuJin Electronics Inc.
10 *
11 *****/
12
13 #include<stdio.h>
14
15 int main()
16 {
17     int x=0x12345678;
18     unsigned char *px=(unsigned char *)&x;
19
20     if(*px == 0x78) {
21         printf("This computer is based of Little Endian\n");
22     } else {
23         printf("This computer is based of Big Endian\n");
24     }
25 }
Memory
Address: px
(e.g. 0x401060, or &v)
0x28ff18: 78 56 34 12 18 ff 28 00
0x28ff20: 94 ff 28 00 fd 10 40 00
0x28ff30: 90 1d 68 00 ff ff ff ff
0x28ff40: b2 e6 1a 12 fe ff ff ff
    
```



✓ C/C++ Pointer 정리 :

pointer관련 symbol	설 명	특 징
<u>*포인터변수명</u>	pointer 변수 선언. 예) <code>int Dat, Dat2, *pDat;</code>	선 언
<u>&일반변수명</u>	변수의 값이 저장된 memory 번지. 예) <code>pDat=&Dat</code>	상 수
<u>*포인터변수명(역참조)</u>	pointer 변수에 저장된 번지에 저장되어 있는 값을 읽을 때. 예) <code>Dat2=*pDat;</code> pointer 변수에 저장된 번지에 새로운 값을 저장할 때. 예) <code>*pDat=Dat2;</code>	변 수

BML_16

[표 2.1-2] 기본적인 C code data types 정리.

변수와 pointer에 대한 학습 - 6.

Ch2_6.c

```

17  /**
18
19  // List all of function prototypes needed
20  void Add(char *aa, char *bb);
21
22  int main()
23  {
24      // In here, add all of needed variables before adding any instructions.
25      char x=100, y=30, z=0;
26
27      // From here, instruction will be added, so we can't add any more variables.
28      Add(&x, &y);
29
30      z=x;
31
32      return 0;
33  }
34
35  void Add(char *aa, char *bb)
36  {
37      char a=*aa, b=*bb, c=0;
38
39      c=a + b;
40      aa=&c;
41  }
42
  
```

Debugger Window (Watch 1):

Name	Value	Type
aa	0x20000654 "d"	char *
bb	0x20000650 ""	char *
a	100 'd'	char
b	30	char
c	130 '?'	char

[그림 2.1-21] C coding 실습 : Ex2_3 (1).

변수와 pointer에 대한 학습 - 7.

```

12 #include<stdio.h>
13 #include<math.h>
14
15 const double PI=3.141593;
16
17 double Deg2Rad(double DegNum);
18 int main()
19 {
20     double DegVal, RadVal;
21     double Value;
22
23     DegVal=45.0; // 45[degree]
24     RadVal=Deg2Rad(DegVal);
25
26     // double sin(double x), [표 3.3.1-4] 참조!
27     Value=sin(RadVal);
28
29     printf("sin(45[degree])= %lf\n", Value);
30
31     return 1;
32 }
33
34 // x[deg]= x * ((1/180)*PI)[Radian];
35 double Deg2Rad(double DegValue)
36 {
37     double Rad;
38
39     printf("입력 매개변수 크기 = %d\n", sizeof(DegValue));
40     Rad=DegValue*((1.0/180)*PI);
41
42     return Rad;
43 }

```

(a) 값에 의한 함수 호출(call by value)

```

12 #include<stdio.h>
13 #include<math.h>
14
15 const double PI=3.141593;
16
17 double Deg2Rad(double *DegNum);
18 int main()
19 {
20     double DegVal, RadVal;
21     double Value;
22
23     DegVal=45.0; // 45[degree]
24     RadVal=Deg2Rad(&DegVal);
25
26     // double sin(double x), [표 3.3.1-4] 참조!
27     Value=sin(RadVal);
28
29     printf("sin(45[degree])= %lf\n", Value);
30
31     return 1;
32 }
33
34 // x[deg]= x * ((1/180)*PI)[Radian];
35 double Deg2Rad(double *DegValue)
36 {
37     double Rad;
38
39     printf("입력 매개변수 크기 = %d\n", sizeof(DegValue));
40     Rad=(*DegValue)*((1.0/180)*PI);
41
42     return Rad;
43 }

```

(b) 번지에 의한 함수 호출(call by reference)

```

C:\Fermi_Room\My201605_BookProject\Sulin_Project\BeginnerC2016
값에 의한 함수 호출(Call by value)

입력 매개변수 크기 = 8
-----
sin(45[degree])= 0.707107

```

(a) 값에 의한 함수 호출(call by value)

```

C:\Fermi_Room\My201605_BookProject\Sulin_Project\BeginnerC2016
번지에 의한 함수 호출(Call by reference)

입력 매개변수 크기 = 4
-----
sin(45[degree])= 0.707107

```

(b) 번지에 의한 함수 호출(call by reference)

✓ 배열의 선언과 정의 :

```

12 #include <stdio.h>
13
14 int main(void) {
15     char CharDats[4]={0x78, 0x56, 0x34, 0x12};
16
17     char CharDats0, CharDats1, CharDats2, CharDats3;
18
19
20     CharDats0=CharDats[0];
21     CharDats1=CharDats[1];
22     CharDats2=CharDats[2];
23     CharDats3=CharDats[3];
24
25     return 1;
26 }

```

배열 선언 및 초기화 ①

원하는_data_type 배열명[정수 상수]= {","로 구분된 각각의 원소 초기 값 나열};

배열 원소 참조 방식 ②

배열명[정수 상수];

- ✓ 배열의 선언과 정의 :

***(pCharDat + 정수)**

pCharDat[정수]

```

12  #include <stdio.h>
13
14  int main(void) {
15      char *pCharDat, CharDats[4]={0x78, 0x56, 0x34, 0x12};
16      char CharDats0, CharDats1, CharDats2, CharDats3;
17      char CharDat0, CharDat1, CharDat2, CharDat3;
18
19      pCharDat=CharDats;
20
21      CharDats0=CharDats[0];
22      CharDats1=CharDats[1];
23      CharDats2=CharDats[2];
24      CharDats3=CharDats[3];
25
26      CharDat0=*(pCharDat++);
27      CharDat1=*(pCharDat++);
28      CharDat2=*(pCharDat++);
29      CharDat3=*(pCharDat);
30
31      pCharDat=pCharDat-3;
32
33      return 1;

```

Watches (new)		
Function arguments		
Locals		
&CharDats[0]	0x28ff10 "xV4W022W022"	char *
pCharDat	0x28ff10 "xV4W022W022"	char *
CharDats0	0x78	char
CharDats1	0x56	char
CharDats2	0x34	char
CharDats3	0x12	char
CharDat0	0x78	char
CharDat1	0x56	char
CharDat2	0x34	char
CharDat3	0x12	char
Memory		
Address: 0x0028ff10		
(e.g. 0x401060, or &variable, or \$eax)		
0x28ff10: 78 56 34 12 12 34 56 78 12 34 56 78 10 ff 28 00		
0x28ff20: 28 ff 28 00 34 9e d6 75 94 ff 28 00 fd 10 40 00		

✓ 배열과 포인터의 관계 :

`int Dat1[2]={0}, *p1Dat`



`p1Dat=Dat1;
p1Dat=&Dat1[0];`

`*(pCharDat + 정수)`



`pCharDat[정수]`

[예 제 4.2-3] pointer symbol *

간혹, 아주 사소한 다음과 같은 내용으로 혼동하는 분들을 볼 수 있다.

- ① `int *a, *b, *c;`
- ② `int * a, * b, * c;`
- ③ `int* a, b, c;`

어쨌든, 함수의 이름, 즉, 함수의 시작 번지를 함수(function) pointer라고 부른다. 다음은 함수 pointer 변수 pfunc를 선언한 예를 보여주고 있다.

```
int (*pfunc) (int, int *);
```

함수 pointer에 대한 학습 - 2.

```
#include<stdio.h>
① int InstArray(int a);
② int DataArray[5]={1,2,3,4,5};
③ int main()
{
    int i, j;
    ④ i=InstArray(2);
    ⑤ j=(&DataArray[0]);
    return 1;
}
⑥ int InstArray(int a)
⑦ {
    int b=0;

    b=a+1;
    return(b);
}
```

① **int InstArray(int a);** → 일반 데이터에 대한 pointer 선언과 정의에 해당

② **int DataArray[5]={1,2,3,4,5};** → 데이터에 대한 pointer 선언과 정의

④ **i=InstArray(2);** → InstArray의 첫 번째 명령어 번지로 분기!

⑤ **j=(&DataArray[0]);** → DataArray의 첫 번째 데이터 번지로 분기!

⑥ **int InstArray(int a)** → Linker에 의해서 첫 번째 명령어의 번지 할당!

Linker에 의해서 첫 번째 데이터 번지 할당!

Ch3Ex0_1.c

함수 pointer에 대한 학습 - 3.

```
#include<stdio.h>
```

① `int (*fp1)(int x);`

함수 pointer의 선언과 NULL pointer로 정의.

```
int (*fp1)(int x)=NULL;
```

② `int InstArray(int a);`

일반 InstArray() 함수 선언 및 정의.

③ `int main()`

```
{
    int i;
```

InstArray()의 첫번째 명령어 번지를 함수 pointer fp1에 저장!

④ `fp1=InstArray;`

⑤ `i=fp1(1);`
`return 1;`

InstArray()의 첫번째 명령어 번지부터 실행!

⑥ `int InstArray(int a)`

Linker에 의해서 첫번째 명령어의 번지 할당!

⑦ `{`
`int b=0;`

```
    b=a+1;
    return(b);
}
```

InstArray() 함수를 명령어 block으로 초기화!

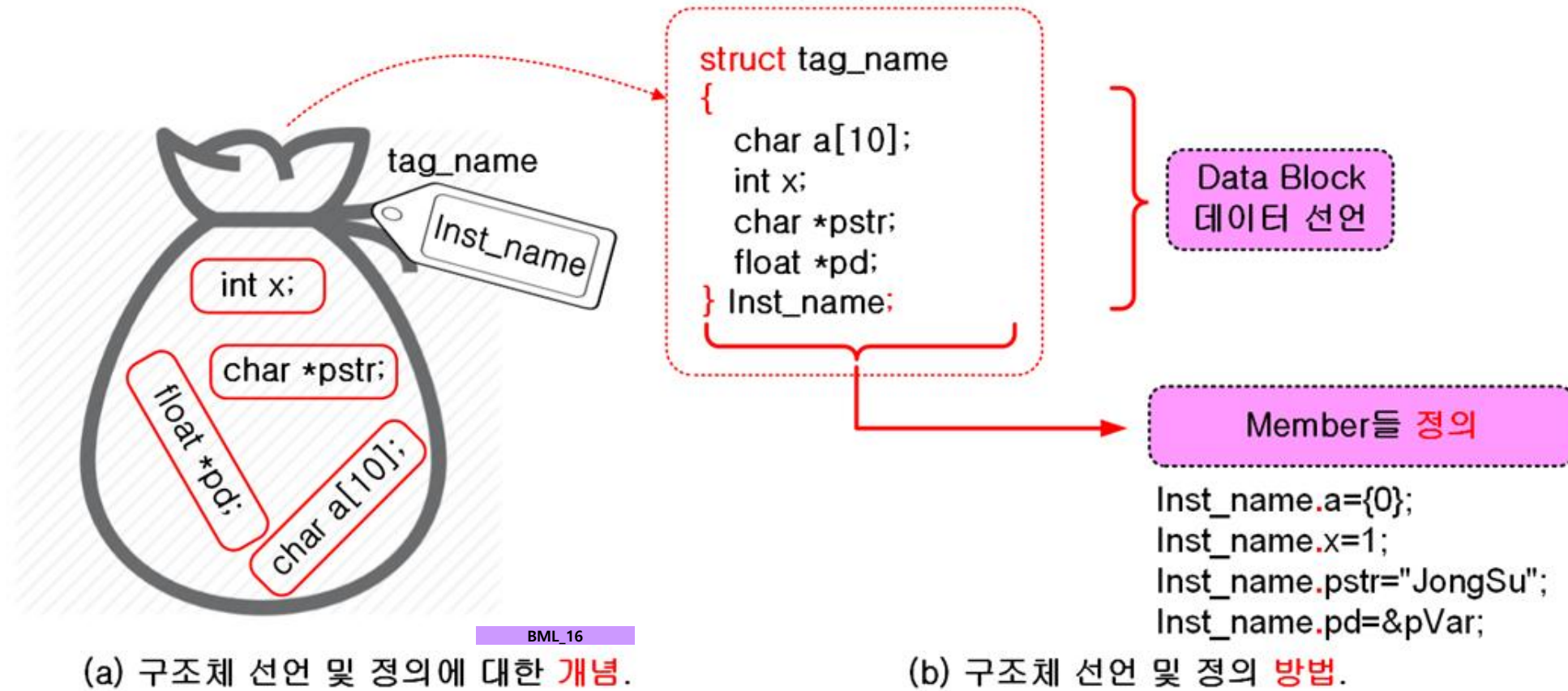
Ch3Ex0_2.c

✓ 문자열의 의미

- 문자 상수(character constant) :
 - 단일 인용부호로 한 개의 문자를 둘러싸은 'a'을 문자 상수라고 하며, 1byte 크기를 가짐.
 - 예 : `char ch1=8, ch2='8';` // `ch1=0x08, ch2=0x38` 즉, ASCII code 값으로 0x38이 문자 8에 해당.
- 문자열 상수(문자열, string) :
 - 동일한 type의 문자 상수들을 여러 개 묶어서 만든 배열(array).
 - 임의의 문자열은 항상, `\0`(즉, null 문자)을 사용하여 문자열의 끝을 알려주어야 한다.
 - C compiler는 이중 인용부호(즉, “와 ”) 사이에 위치한 것은 일반 문자, 특수 문자 또는 공백 문자 등에 상관하지 않고 무엇이든지 문자열로 취급.
 - 이중 인용부호를 사용하여 문자열을 정의하는 경우, 자동으로 C compiler가 null 문자를 삽입.
 - 배열은 symbol `[, ]`을 사용!

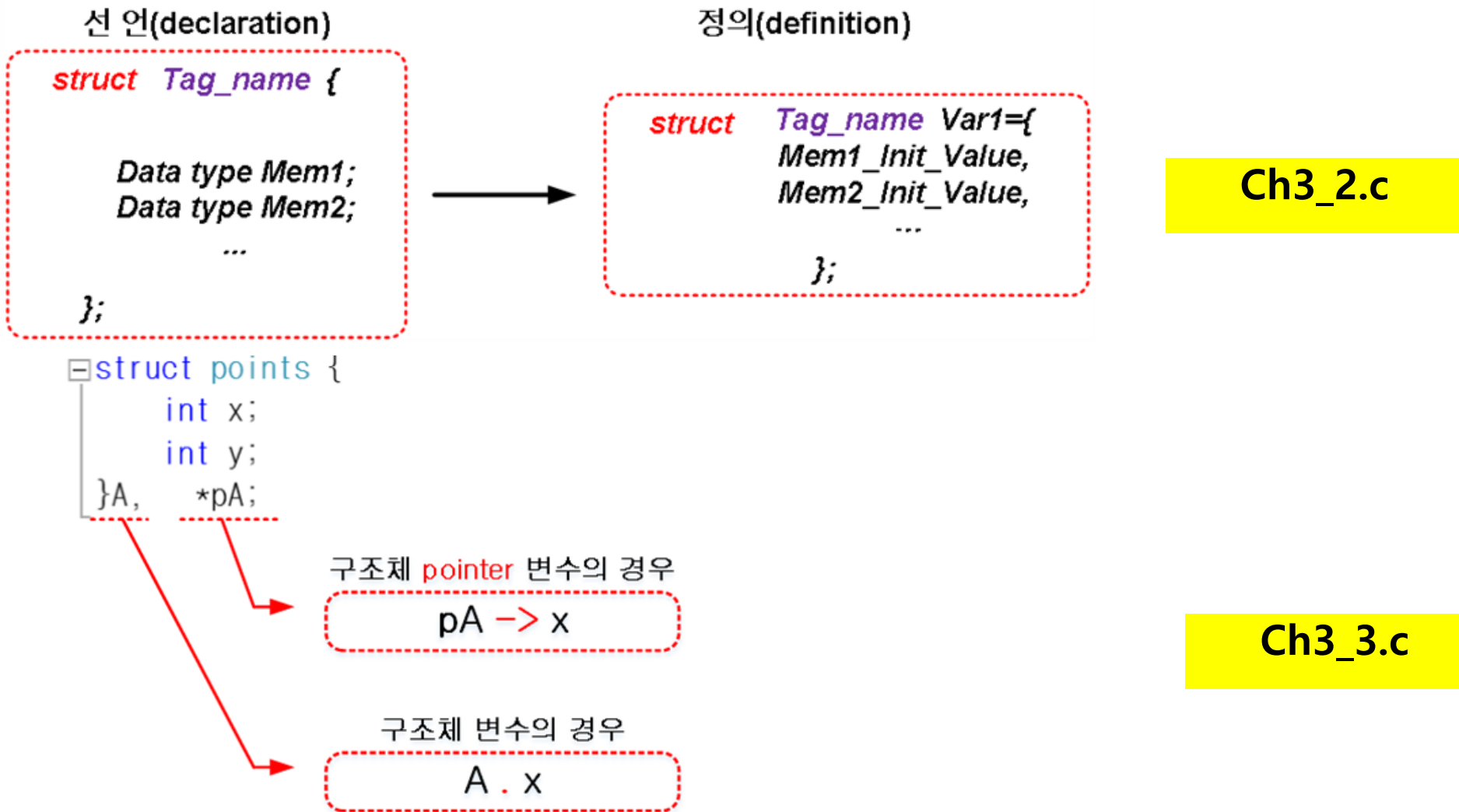


구조체에 대한 학습 - 1.



BML_16

구조체에 대한 학습 - 2.

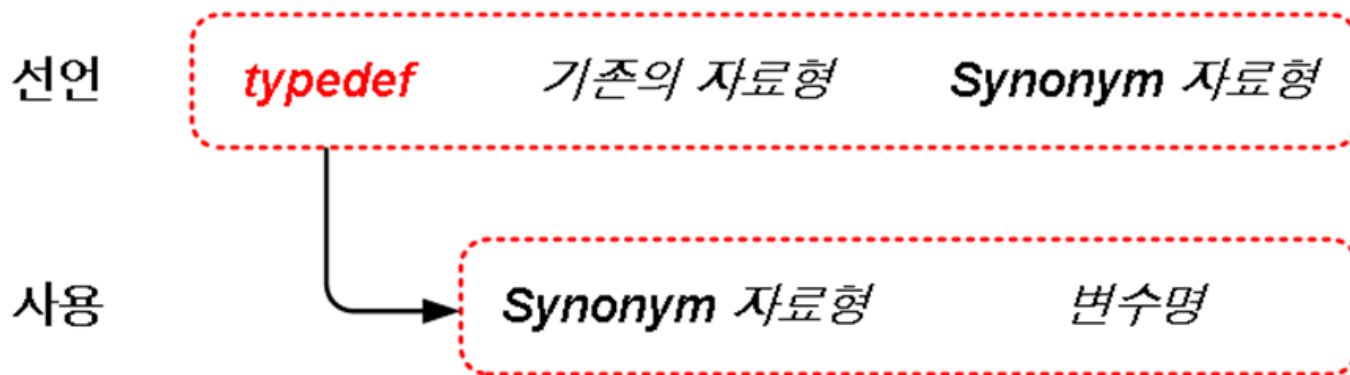


[그림 6.1-5] 구조체 변수의 member 접근 방법.

BML_17

✓ typedef 의미 :

- 새로운 자료형을 만드는 것이 아니라, 기존의 자료형 이름 외에 편하게 사용할 수 있는 새로운 이름을 추가로 만들 때 사용 하는 keyword.
- 예 : typedef unsigned char bool로 재정의 하였더라도 계속해서 unsigned char temp;와 같이 사용할 수 있다.
- typedef는 C언어에서 제공하는 기본적인 자료형과 이전 typedef에서 재정의한 자료형 이름에만 사용할 수 있다.



[그림 9.1.2-3] typedef keyword 사용 방법.

✓ enum data type 의미 :

- 구조체와 동일한 모양을 가지는데, 단지, member가 숫자에 대응한다는 것만 다르다.

예 : #include<stdio.h>

```
enum {  
    TCP, // 0  
    RTU  // 1  
};  
  
int main() {  
    int x = 0; // int RTU=3;  
    x = RTU;   // 변수 x의 값은 "1"이 된다.  
    return 0;  
}
```

✓ 전역 변수, 지역 변수

Ex1.c

```
#include "Sub.h"
extern int Var;

int main()
{
    int x=1,y=2,z;
    z=subfunc(x,y)+Var;
    return 0;
}
```

Ex2.c

```
int Var = 1;
```

Ex3.c

```
extern int Var;
int subfunc(int x, int y)
{
    return (x+y+Var);
}
```

Sub.h

```
extern int subfunc(int x, int y);
```

Ch3Ex4_1.c
Ch3Ex4_2.c
Ch3Ex4_3.c
Error 발생!!!

✓ 전역 변수, 지역 변수

변수 부류		설 명
지역 변수	자동 변수	선언된 code block과 그 하위 code block에서만 유효. keyword auto 사용. 그러나, 대부분 생략하여 사용함.
	register 변수	core registers를 이용하도록 한다. read/write 속도가 제일 빠르 지만 pointer 선언이 불가능하다.
	정적 변수	선언된 code block과 그 하위 code block에서만 유효하지만 program 종료 시까지 계속해서 값을 유지. keyword static 사용.
전역 변수	전역 변수	전역 변수로 선언한 데이터는 함수 사이 또는 source file들에 사 이에 공유가 가능하며, program 종료 시까지 계속해서 값을 유지. 단, source file들 사이에 공유하기 위해서는 keyword extern 사 용. 그러나, 초기화가 한 번만 있다면 생략가능.

[표 4.3.2-1] C 언어의 변수에 대한 정리.

- ✓ 모든 파일은 `fopen()` 함수로 file을 열고, 반드시 `fclose()` 함수로 닫아주어야 한다는 데 주의.

전형적인 사용방법(Syntax) : `fopen()`

a) `FILE *fopen(char *name, char *mode);`

- ▶ 표준 C library인 `stdio.h`에 속해 있다.
- ▶ 첫 번째 입력 매개변수 `name`은 파일 이름을 저장하고 있는 문자열이다.
- ▶ 두 번째 입력 매개변수 `mode`는 지정한 파일을 처리하는 방법으로서 [표 7.1-1]에서 나열한 문자열 중에서 하나를 지정하면 된다. 단, 문자가 아닌 문자열인데 주의!
- ▶ 출력 매개변수인 `FILE` 구조체의 반환 값이 `NULL`이 아니면 성공이다.

mode	설 명
"w"	쓰기 작업을 위해서 파일을 연다. 만일, 동일한 이름의 파일이 있으면 내용을 모두 지운다.
"r"	읽기 작업을 위해서 파일을 연다. 만일, 동일한 이름의 파일이 없으면 <code>NULL</code> 을 반환.
"a"	추가 작업을 위해서 파일을 연다. 만일, 동일한 이름의 파일이 없으면 새로 생성.

[표 7.1-1] File Mode 문자열.

```
#include<stdio.h>
#include<stdint.h>
#include<math.h>

#define SamplesNum      100
#define PI              3.141592653589793
uint16_t DACSinDat[100] = {0,};

int main() {
    FILE *fp = NULL;
    int32_t i = 0, j = 0;
    double t=0, y = 0;
    uint16_t x1 = 0xFFFF, x4=0;
    int16_t x2 = -1, x3=0xFFFF-1;
    x4 = (uint16_t)x3;
```

```
    fp = fopen("DACDat.txt","w");
    for(i=0;i<SamplesNum;i++) {
        y = 2047+2047*sin(2*PI*t);
        t += 0.01;                      // sampling time for
        1 period.
        DACSinDat[i] = (uint16_t)(y+0.5); // to make it a fixed
        integer.
        fprintf(fp,"%d\\n",DACSinDat[i]);
    }
    fclose(fp);

    return 0;
}
```

C header file 작성 기준.

header file에 포함되어야 할 items	구조체 정의	struct
	전역함수 선언	char func();
	전역변수 선언	extern int a;
	나열자	enum
	macro 및 inline 함수 정의	#define __inline int func() {...};
	type 정의	typedef
header file에 포함되지 말아야 하는 items	전역함수/전역변수 정의	char func() {...}; / int a;
	const 전역변수의 정의	const int size=1024;

BML_18

C 언어의 Keyword.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while